AI Interactive Storyteller

Stage 2 Implementation Report

EECS 3311 Software Design, Section A, Fall 2026
Shane Marcelo
221114749
https://github.com/Polarralop/EECS3311Project
November 10th, 2026

Contents

1. Overview and Implementation Status....3
1.1 Project Summary....3
1.2 Implementation Status....5
1.3 LLM Integration and Agent Behaviour....6
1.4 Architecture as Built....6
1.5 Known Limitations....7
2. Feature-to-Implementation Mapping....7
F01: Scenario Setup....7
F02: Lore Entry Management....9
F03: Character Management....11
F04: Story Turn....12
F05: Pivotal Events....15
F06: Plot Steering....16
F07: Lore Proposals....18
F08: Consistency Check....20
F09: Story Memory Summarization....22
F10: Undo / Retry / Edit Turn....23
F11: Save and Load Session....25
F12: Ending Generation....27
3. Design Patterns....29
3.1 Adapter....29
3.2 Command....30
3.3 Facade....31
3.4 MVC....32
3.5 Observer....33
3.6 Strategy....34
3.7 Updated UML Class Diagram....34
3.8 Updated Use-Case and Sequence Diagrams....42
4. Changes from Stage 1 and Rationale....53
4.1 Summary of Changes....53
4.2 Change Details....56
5. AI Tools and Collaboration Log....72
5.1 Tools and Models Used....72

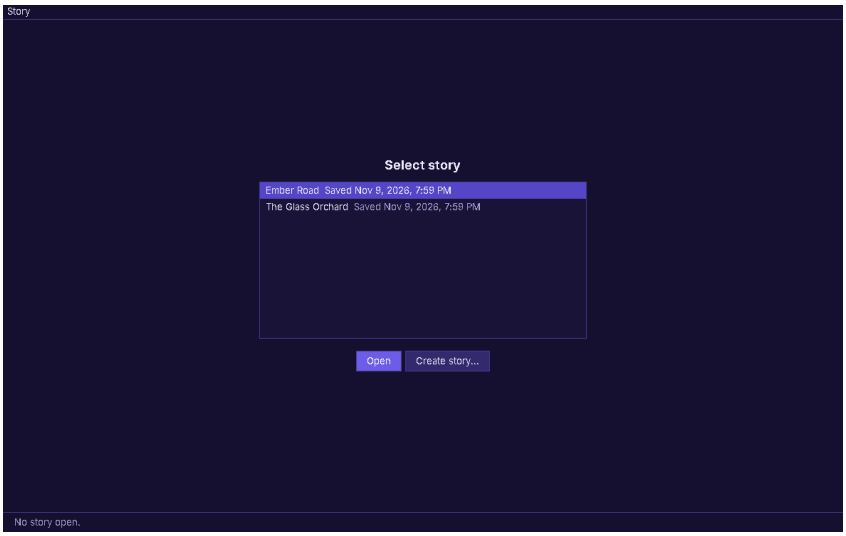
5.2 Cost Summary..... 73
5.3 Representative Interactions..... 73
5.4 How AI Output Was Checked..... 74
6. Development and Verification..... 74
6.1 Weekly Commit Summary..... 74
6.2 Checks Performed..... 79
6.3 Defects Found and Fixed..... 82
6.4 Limitations and Stage 3 Work..... 82
7. Demonstration Index..... 82

1. Overview and Implementation Status

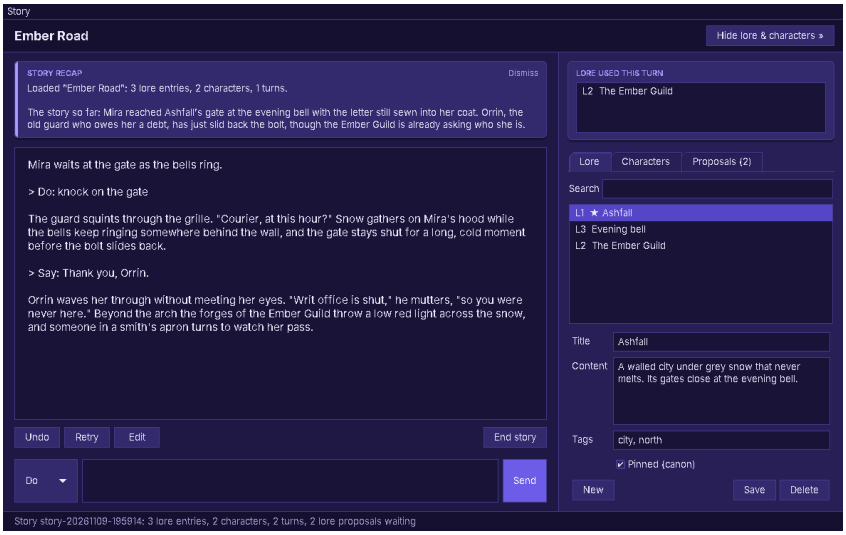
1.1 Project Summary

The AI Interactive Storyteller is a desktop application for solo role-players who want to write a long story together with an AI storyteller. The user sets up a scenario (or has the agent draft one from a short idea), keeps a lore book and a cast of characters, and then plays turn by turn in three input modes: Do (an action), Say (dialogue) and Story (the user writes part of the story and the agent continues it). Behind every turn the agent keeps the story consistent with the lore, steers it towards a hidden plan of pivotal events that the user never sees, proposes new lore for the user to approve, summarizes older turns so a long story fits in its context, and writes an ending when the plan is complete or the user asks for one. Turns can be undone, retried or edited, and stories are saved and loaded as JSON with a model-written recap on load.

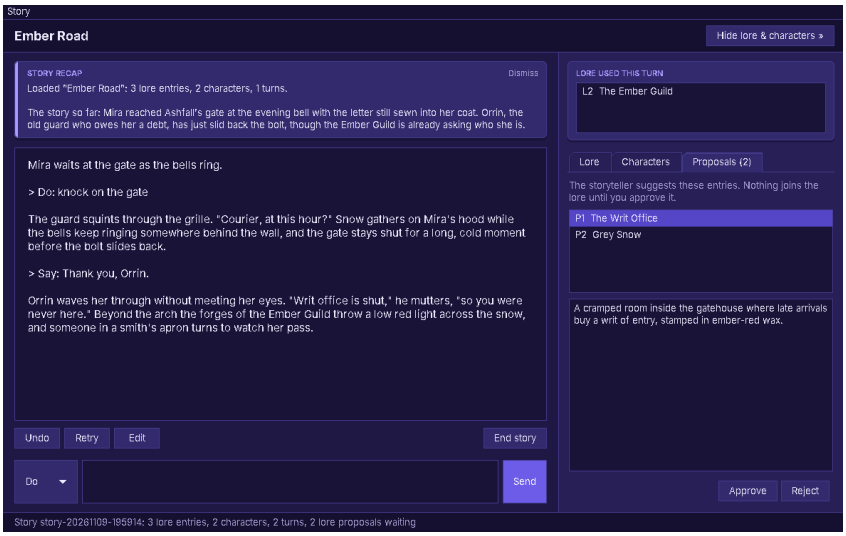
Stage 2 implements the Stage 1 design in Java 21 with Maven: all 12 Stage 1 features (F01 to F12) and all six designed patterns (Adapter, Command, Facade, MVC, Observer, Strategy) are in the code, and every feature is reachable from both front ends, a Swing GUI (StoryGUI) and a command-line interface (StoryCLI), which share one StoryController. Where the implementation differs from the Stage 1 design, the difference is recorded in Section 4. Screenshots 1 to 3 show the GUI as submitted.



Screenshot 1. The GUI's start screen: saved stories listed by title with their save time (F11).



Screenshot 2. A loaded story: the recap card (F09), the transcript with Do and Say turns (F04), Undo, Retry, Edit and End story (F10, F12), and the sidebar with the lore used this turn and the Lore tab (F02).



Screenshot 3. The Proposals tab of the sidebar, where the user approves or rejects lore the agent proposed (F07).

Screenshots 2 and 3 were rendered from the real StoryGUI with scripted model replies (FakeLLMClient); the demonstration videos (Section 7) show the app with the live model.

1.2 Implementation Status

As of Nov 10, 2026: all 12 Stage 1 features (F01 to F12) work end to end from both the GUI and the CLI. Two additions were made at the user's request (gradual display of new story text, and a GUI start screen with a collapsible sidebar and a new palette); they change how features are shown, not what they do, and are recorded in Section 4. No feature was dropped or replaced.

ID	Feature	Type	Status	GUI	CLI
F01	Scenario Setup	Hybrid	Done	Yes	Yes
F02	Lore Entry Management	Deterministic	Done	Yes	Yes
F03	Character Management	Deterministic	Done	Yes	Yes
F04	Story Turn	Hybrid	Done	Yes	Yes
F05	Pivotal Events	Hybrid	Done	Yes	Yes
F06	Plot Steering	AI	Done	Yes	Yes
F07	Lore Proposals	Hybrid	Done	Yes	Yes
F08	Consistency Check	Hybrid	Done	Yes	Yes
F09	Story Memory Summarization	AI	Done	Yes	Yes
F10	Undo / Retry / Edit Turn	Hybrid	Done	Yes	Yes
F11	Save and Load Session	Deterministic	Done	Yes	Yes
F12	Ending Generation	AI	Done	Yes	Yes

Status values: Done, Partial, Changed (see Section 4), Dropped (reason in Section 4). GUI and CLI: Yes when the feature is reachable from that front end.

1.3 LLM Integration and Agent Behaviour

The application uses Claude Sonnet 5.5 (model id claude-sonnet-5-5, ClaudeAdapter.DEFAULT_MODEL, overridable with STORYTELLER_MODEL in .env). It is reached only through the LLMClient interface: ClaudeAdapter is the one class that imports the anthropic-java SDK, and it turns an LLMRequest (system prompt, LLMMessage list, LLMToolSpec list) into a Messages API call and the reply into an LLMResponse (text, tool calls, refusal flag, token counts). The prompts are plain text files under src/main/resources/prompts/. Tests use FakeLLMClient, so no test makes a paid call.

The model acts as an agent, not as prompt-in/text-out:

Tool use. A story turn (StoryAgent.takeTurn) is a loop of up to 8 rounds (StoryAgent.MAX_TOOL_ROUNDS). In each round the model may call four tools that the app validates and executes through ToolRegistry: search_lore (SearchLoreTool, retrieval over the lore book), get_character (GetCharacterTool), propose_lore (ProposeLoreTool, which raises a LoreProposal the user must approve) and update_plot (UpdatePlotTool, which marks a pivotal event reached or revises the pending ones). The model never changes state directly; an invalid call comes back to it as a failed ToolResult.

Planning. Before a story starts, PlotPlanner.createPlot asks the model for a hidden plan of pivotal events (at most 8); the story does not start if planning fails. Each turn, PlotPlanner.planStep turns the plan into private guidance for the narrator, and the model decides through update_plot when an event has happened or the plan must change (F05, F06). The plan is never shown in either front end.

Memory. MemoryManager keeps the last 8 turns verbatim and has the model summarize older turns into a running summary at the start of a turn (F09); the summary and recent turns form the context of each turn, and a recap is written when a story is loaded.

Checking and deciding. After the narration, ConsistencyChecker asks the model whether the new text contradicts any pinned (canon) lore entry or the lore the turn looked up, and attaches up to 5 warnings without blocking the turn (F08). When the plan completes, the agent offers to end the story and StoryAgent.generateEnding writes the ending from the plan, summary and recent turns (F12).

1.4 Architecture as Built

The code follows the Stage 1 package structure under ca.yorku.storyteller: ui (StoryGUI, StoryCLI, StoryObserver, plus TranscriptPacer and StoryTheme), controller (StoryController, CommandHistory, StoryCommand, SubmitTurnCommand, EditTurnCommand, SavedStory), model (StorySession and everything it owns: Scenario, LoreBook, LoreEntry, Turn, LoreProposal, Plot, PivotalEvent, Character, Relationship and two status enums), agent (the StoryAgent facade, PlotPlanner, MemoryManager, ConsistencyChecker and the three InputModeStrategy classes), agent.tools (ToolRegistry, AgentTool and the four tools), llm (LLMClient, ClaudeAdapter, FakeLLMClient and the request, response and message types) and persistence (SessionRepository, JsonSessionRepository). Figure 1 shows every class.

A request flows the same way from either front end. For a story turn, the view reads the input and mode and calls StoryController.submitTurn(input, mode). The controller wraps the call in a SubmitTurnCommand and runs it through CommandHistory; the command calls StoryAgent.takeTurn, which summarizes, builds the context, plans the step, runs the tool loop through ToolRegistry and

LLMClient, and checks consistency. The command adds the resulting Turn to the StorySession, which notifies its observers (StoryGUI and StoryCLI through StoryObserver.onStoryChanged), and each view redraws itself. Lore, character and save/load actions follow the same path without the agent.

Differences from Stage 1 section 1.5, all logged in Section 4: the llm package gained structured message and tool types (LLMMessage, LLMToolCall, LLMToolResult, LLMToolSpec) and LLMException; the ui package gained TranscriptPacer (gradual display of new text) and StoryTheme (the GUI's palette); the controller gained the SavedStory record for the start screen; errors travel to the views as exceptions instead of the controller calling showError.

1.5 Known Limitations

All 12 features work end to end, with these limitations:

Automated tests script the model with FakeLLMClient. They check how the app handles model replies, including refusals, failures and bad tool calls, but not the quality of the storytelling itself, which depends on the live model.

Turns are not streamed. A turn can take several model calls (tool rounds and the consistency check), and the text appears only when the turn is complete; it is then revealed gradually on screen. This was chosen so that a failed or retried turn never shows text that is then taken back (Section 4).

The consistency check and the plot steering rely on the model's judgement: a contradiction with pinned lore can be missed, a warning can be a false alarm, and the model decides when a pivotal event has been reached.

Older turns are kept only as a summary once a story passes 8 turns, so fine details of early turns can be lost.

Undo history is not saved (as designed): after a load, undo and retry start empty; editing the last turn still works.

If the tool loop needs more than 8 rounds the turn fails with a message and nothing changes; the user can send it again.

Saves are local JSON files for one user; there is no locking if the same story is open in two running copies of the app.

Full deterministic testing and KUMA agent validation are Stage 3 work (Section 6.4).

2. Feature-to-Implementation Mapping

For each Stage 1 feature: its Stage 1 reference, implementation status, the classes and files that implement it, the key methods, the patterns involved, how the classes collaborate from the user's input to the result, and where it is shown in the demonstration videos.

F01: Scenario Setup

Feature ID and Name	F01: Scenario Setup
Stage 1 Reference	Stage 1 report section 2.2 and section 9 (F01); UC01 Create a story; SD01 Generate a scenario; SD02 Start a story
Implementation Status	Done. Reachable from the GUI (Story > Start story...) and the CLI (start).

Type	Hybrid, as in Stage 1. Drafting calls the model (StoryAgent.generateScenario); checking the fields, filling in a missing title and creating the story are deterministic code (Scenario.readyToStart, StoryController.startStory).
Classes / Files and Responsibilities	StoryGUI (src/main/java/ca/yorku/storyteller/ui/StoryGUI.java): Start story form with idea + Generate draft, Title, Premise, Tone, Setting, Opening situation, Plot basis and Start story; runs model calls in a SwingWorker. StoryCLI (src/main/java/ca/yorku/storyteller/ui/StoryCLI.java): start asks for an idea, shows the draft, then asks for each field with the draft as the default. StoryController (src/main/java/ca/yorku/storyteller/controller/StoryController.java): generateScenario returns a draft without changing anything; startStory validates, plans (F05) and opens the story. StoryAgent (src/main/java/ca/yorku/storyteller/agent/StoryAgent.java): generateScenario sends the idea with prompts/scenario.txt and reads the JSON reply (jsonObjectIn). Scenario (src/main/java/ca/yorku/storyteller/model/Scenario.java): the five fields; readyToStart checks premise and setting and makes a title from the premise. LLMClient, ClaudeAdapter (src/main/java/ca/yorku/storyteller/llm/): carry the call to Claude.
Key Methods	StoryController.generateScenario(String), StoryAgent.generateScenario(String), StoryAgent.jsonObjectIn(String), StoryController.startStory(Scenario, String), Scenario.readyToStart(), Scenario.describe(), StoryGUI.generateDraft(), StoryGUI.startStory().
Design Patterns	MVC: the form and the CLI collect fields and call the controller. Facade: the controller only calls StoryAgent. Adapter: the draft request goes through LLMClient. Observer: the views refresh when the story opens.
Collaboration (Input to Result)	1. GUI: the user types an idea and clicks Generate draft (CLI: start, then the idea). 2. The view calls StoryController.generateScenario(idea), which calls StoryAgent.generateScenario(idea). 3. The agent sends the idea with the scenario prompt through LLMClient.generate and reads title, premise, tone, setting and openingSituation from the JSON reply into a Scenario. 4. The draft fills the form (CLI: is printed and each field is asked again with the draft as the default); the user edits it, or writes everything by hand without a draft. 5. The user clicks Start story; the view builds a Scenario and calls StoryController.startStory(scenario, plotBasis). 6. The controller calls scenario.readyToStart(), has the plan made (F05), puts the scenario on the open empty session or a new one, and notifies the observers; the story view opens on the title and opening situation.

GUI and CLI Access	GUI: Create story... on the start screen, or Story > Start story... (form window). CLI: start.
Error / Alternative Behaviour	Empty idea: "Describe the story you want in a few words first." Claude declines: a message to rephrase or write it by hand. Failed call or a draft without premise or setting: a message; the form keeps what was typed. Empty premise or setting at start: "A story needs a premise." / "A story needs a setting.", no model call. Draft not wanted: the user edits it or asks again (GUI), or answers n (CLI).
Video Link and Timestamp	*[TO ADD: video name, link and mm:ss, matching Section 7]*
Changes from Stage 1	Starting keeps lore and characters prepared in an empty session; the controller takes the agent (Section 4).

F02: Lore Entry Management

Feature ID and Name	F02: Lore Entry Management
Stage 1 Reference	Stage 1 report section 2.2 and section 9 (F02); UC02 Manage lore; SD03 Manage lore entries and characters
Implementation Status	Done. Reachable from the GUI (Lore tab of the sidebar) and the CLI (lore commands).
Type	Deterministic, as in Stage 1. No step calls the model.
Classes / Files and Responsibilities	StoryGUI (src/main/java/ca/yorku/storyteller/ui/StoryGUI.java): Lore tab of the collapsible sidebar with search box, entry list (pinned marked with a star) and entry form. StoryCLI (src/main/java/ca/yorku/storyteller/ui/StoryCLI.java): lore list, search, show, add, edit, pin, unpin and remove commands. StoryController (src/main/java/ca/yorku/storyteller/controller/StoryController.java): adds or updates an entry, removes, searches and returns copies for editing; notifies observers. StorySession (src/main/java/ca/yorku/storyteller/model/StorySession.java): owns the lore book and notifies the front ends. LoreBook (src/main/java/ca/yorku/storyteller/model/LoreBook.java): stores entries, assigns ids L1, L2, ..., rejects duplicate ids and titles, searches with pinned entries first, returns the pinned entries. LoreEntry (src/main/java/ca/yorku/storyteller/model/LoreEntry.java): one entry; validates title, content and tags; matches(query).

Key Methods	StoryController.saveLoreEntry(LoreEntry), StoryController.removeLoreEntry(String), StoryController.searchLore(String), StoryController.getLoreEntry(String), LoreBook.add(LoreEntry), LoreBook.update(LoreEntry), LoreBook.remove(String), LoreBook.search(String), LoreBook.getPinned(), LoreEntry.matches(String), LoreEntry.setTags(List<String>), StorySession.notifyObservers(), StoryObserver.onStoryChanged(StorySession).
Design Patterns	MVC: the views build a LoreEntry from the form and call StoryController; LoreBook holds the rules. Observer: after each change StorySession.notifyObservers() refreshes the GUI lists and prints the CLI status line.
Collaboration (Input to Result)	1. The user fills the Lore form and clicks Save (GUI), or types lore add and answers the prompts (CLI). 2. The front end builds a LoreEntry (its constructor rejects an empty title or content and invalid tags) and calls StoryController.saveLoreEntry(entry). 3. The controller calls LoreBook.update(entry) if the id is already stored, otherwise LoreBook.add(entry), which assigns the next id and rejects a duplicate title. 4. The controller calls StorySession.notifyObservers(); StoryGUI.onStoryChanged re-reads the list through searchLore and StoryCLI.onStoryChanged prints the new counts. 5. Search: the GUI search box or lore search <words> calls StoryController.searchLore(query); LoreBook.search keeps the entries whose matches(query) is true, pinned first.
GUI and CLI Access	GUI: Lore tab of the sidebar on the right of the story (search box, list, Title / Content / Tags / Pinned fields, New, Save, Delete; the sidebar collapses with the button in the heading). CLI: lore [list], lore search <words>, lore show <id>, lore add, lore edit <id>, lore pin <id>, lore unpin <id>, lore remove <id>.
Error / Alternative Behaviour	An empty title or content, an invalid tag (not one word of letters, digits or hyphens), a duplicate title, an unknown id, or no open story: the model or controller throws with a readable message, nothing is stored, and the GUI shows it in a dialog (showError) while the CLI prints "Error: ...".
Video Link and Timestamp	*[TO ADD: video name, link and mm:ss, matching Section 7]*
Changes from Stage 1	Errors are returned to the front end as exceptions instead of the controller calling showError; added removeLoreEntry and getLoreEntry (Section 4, "Controller: errors travel as exceptions").

F03: Character Management

Feature ID and Name	F03: Character Management
Stage 1 Reference	Stage 1 report section 2.2 and section 9 (F03); UC03 Manage characters; SD03 Manage lore entries and characters
Implementation Status	Done. Reachable from the GUI (Characters tab of the sidebar) and the CLI (char and rel commands).
Type	Deterministic, as in Stage 1. No step calls the model.
Classes / Files and Responsibilities	StoryGUI (src/main/java/ca/yorku/storyteller/ui/StoryGUI.java): Characters tab of the collapsible sidebar with the character list, the form and a relationships section (list, target, type, notes, Add, Remove selected). StoryCLI (src/main/java/ca/yorku/storyteller/ui/StoryCLI.java): char list, show, add, edit, remove; rel add, rel remove. StoryController (src/main/java/ca/yorku/storyteller/controller/StoryController.java): saves a new or edited character, removes, adds and removes relationships, returns copies for editing. StorySession (src/main/java/ca/yorku/storyteller/model/StorySession.java): stores characters, finds them by name ignoring case, rejects duplicates, checks relationship targets, follows renames and removals. Character (src/main/java/ca/yorku/storyteller/model/Character.java): name, description, traits, one-way relationships; describe() is the profile the agent will receive. Relationship (src/main/java/ca/yorku/storyteller/model/Relationship.java): target name, type and notes.
Key Methods	StoryController.saveCharacter(String originalName, Character), StoryController.removeCharacter(String), StoryController.addRelationship(String, String, String, String), StoryController.removeRelationship(String, String), StorySession.addCharacter(Character), StorySession.updateCharacter(String, Character), StorySession.removeCharacter(String), StorySession.addRelationship(String, Relationship), StorySession.findCharacter(String), Character.addRelationship(Relationship), Character.describe().
Design Patterns	MVC and Observer, as in Stage 1: the views forward to StoryController, the rules are in StorySession and Character, and notifyObservers() refreshes both front ends.

Collaboration (Input to Result)	1. The user fills the character form and clicks Save character (GUI), or types char add / char edit <name> (CLI). 2. The front end calls StoryController.saveCharacter(originalName, character), with originalName null for a new character. 3. StorySession.addCharacter or updateCharacter rejects a duplicate name and checks every relationship target; a rename rewrites the relationships that point at the old name. 4. For a relationship, the front end calls StoryController.addRelationship(from, target, type, notes); StorySession checks the target exists and is not the same character, then Character.addRelationship stores it one way. 5. The controller calls notifyObservers(); the GUI refreshes the list and relationship targets and the CLI prints the counts.
GUI and CLI Access	GUI: Characters tab of the sidebar (list; Name, Description, Traits; Relationships list with To / Type / Notes, Add and Remove selected; New, Save, Delete character). CLI: char [list], char show <name>, char add, char edit <name>, char remove <name>, rel add, rel remove.
Error / Alternative Behaviour	Empty name, a name already used by another character, a relationship to a missing character or to itself, or an unknown character: the operation is refused with a message (dialog in the GUI, "Error: ..." in the CLI) and nothing changes. Removing a character also removes relationships pointing at it.
Video Link and Timestamp	*[TO ADD: video name, link and mm:ss, matching Section 7]*
Changes from Stage 1	Relationship stores the target by name; StorySession gains updateCharacter, removeCharacter and addRelationship; saveCharacter takes the original name (Section 4, "F03: Relationships point at a character by name" and "Controller").

F04: Story Turn

Feature ID and Name	F04: Story Turn
Stage 1 Reference	Stage 1 report section 2.2 and section 9 (F04); UC04 Take a story turn; SD04 Submit a story turn; SD05 The agent narrates the turn
Implementation Status	Done. Reachable from the GUI (the story view) and the CLI (do, say, story).
Type	Hybrid, as in Stage 1. The narration and the choice of lookups come from the model; framing the input, building the context, validating and running every tool call, recording the lore used and adding the turn are deterministic code.

Classes / Files and Responsibilities	StoryGUI (src/main/java/ca/yorku/storyteller/ui/StoryGUI.java): the story column with the transcript, mode selector, input box and Send, and the "Lore used this turn" card at the top of the sidebar; the turn runs in a SwingWorker. StoryCLI (src/main/java/ca/yorku/storyteller/ui/StoryCLI.java): do / say / story <text>, turns; prints the narration and "(Lore used: …)". StoryController (src/main/java/ca/yorku/storyteller/controller/StoryController.java): submitTurn picks the InputModeStrategy by name and runs a SubmitTurnCommand. CommandHistory, SubmitTurnCommand (src/main/java/ca/yorku/storyteller/controller/): run the turn as an undoable command. InputModeStrategy, DoModeStrategy, SayModeStrategy, StoryModeStrategy (src/main/java/ca/yorku/storyteller/agent/): frame the input. StoryAgent (src/main/java/ca/yorku/storyteller/agent/StoryAgent.java): takeTurn builds the prompt and runs runToolLoop. MemoryManager (src/main/java/ca/yorku/storyteller/agent/MemoryManager.java): buildContext (summary + last 8 turns). PlotPlanner (src/main/java/ca/yorku/storyteller/agent/PlotPlanner.java): planStep (next pending event as hidden guidance). ToolRegistry, AgentTool, ToolCall, ToolResult, SearchLoreTool, GetCharacterTool, ProposeLoreTool (src/main/java/ca/yorku/storyteller/agent/tools/): validate and run the tool calls. prompts/system.txt (src/main/resources/prompts/system.txt): the narrator instructions. StorySession, Turn (src/main/java/ca/yorku/storyteller/model/): hold the result.
Key Methods	StoryController.submitTurn(String, String), CommandHistory.run(StoryCommand), SubmitTurnCommand.execute(), StoryAgent.takeTurn(StorySession, String, InputModeStrategy), InputModeStrategy.frame(String), MemoryManager.buildContext(StorySession), PlotPlanner.planStep(StorySession, String), StoryAgent.runToolLoop(LLMRequest, StorySession, Set<String>), LLMClient.generate(LLMRequest), ToolRegistry.execute(ToolCall, StorySession), AgentTool.execute(Map, StorySession), StorySession.addTurn(Turn), StoryGUI.submitInput().

Design Patterns	All six, as in Stage 1. Strategy: the mode object frames the input. Command: the turn is a SubmitTurnCommand run by CommandHistory. Facade: StoryAgent hides the planner, memory, tools and model. Adapter: every model call goes through LLMClient. MVC: the views only forward and display. Observer: StorySession notifies the views after the turn.
Collaboration (Input to Result)	1. The user picks Do, Say or Story, types the input and clicks Send (CLI: do / say / story <text>). 2. The view calls StoryController.submitTurn(input, modeName); the controller selects the InputModeStrategy and runs a SubmitTurnCommand through CommandHistory.run. 3. SubmitTurnCommand.execute copies the plot and calls StoryAgent.takeTurn(session, input, mode). 4. The agent frames the input with mode.frame(input) and adds the scenario, the known character names, MemoryManager.buildContext(session) and the hidden guidance from PlotPlanner.planStep. 5. runToolLoop calls LLMClient.generate with the search_lore, get_character and propose_lore tools. While the reply has tool calls, each becomes a ToolCall run by ToolRegistry.execute, and the ToolResults go back to the model; lore ids from successful lookups are collected. 6. When the model answers with text, the agent returns a Turn holding the input, mode, narration and lore used. 7. The command adds it with StorySession.addTurn, the controller notifies the observers, and the views show the narration and the lore used.
GUI and CLI Access	GUI: the story column (mode selector, input box, Send; double-click a "Lore used this turn" entry in the sidebar to open it in the Lore tab). CLI: do <action>, say <words>, story <text>; turns prints the story so far. A new turn appears gradually in the transcript (TranscriptPacer, 300 characters a second, at most 10 seconds; a click shows the rest) and the CLI prints it at the same pace. The heading above the transcript names the story.
Error / Alternative Behaviour	Empty input: a message, nothing sent. No story started: the input box and Send are disabled (CLI: "Start the story first."). Claude declines, the call fails, the narration is empty or the model keeps calling tools for 8 rounds: a message, no turn is added, the plot is put back and the input stays in the box. Invalid tool request (unknown tool, missing argument, unknown character): a failed ToolResult goes back to the model and the turn continues. No relevant lore: the turn continues and the lore-used list is empty.
Video Link and Timestamp	*[TO ADD: video name, link and mm:ss, matching Section 7]*

Changes from Stage 1	Tool types gain descriptions, call ids and lore ids; planStep builds guidance without a model call; the loop is capped at 8 rounds (Section 4). New story text is revealed gradually in both front ends (Section 4, "New: New story text is revealed gradually").

F05: Pivotal Events

Feature ID and Name	F05: Pivotal Events
Stage 1 Reference	Stage 1 report section 2.2 and section 9 (F05); UC01 Create a story; SD02 Start a story
Implementation Status	Done. Planned from the Start story form (GUI) and the start command (CLI); the plan is never displayed in either.
Type	Hybrid, as in Stage 1. The model writes the events (PlotPlanner.createPlot); code validates the scenario first, parses the JSON reply, numbers the events E1, E2, ... as PENDING, caps the plan at 8 and stores it on the session.
Classes / Files and Responsibilities	StoryController (src/main/java/ca/yorku/storyteller/controller/StoryController.java): startStory validates, asks the agent for the plan and opens the story. StoryAgent (src/main/java/ca/yorku/storyteller/agent/StoryAgent.java): planPlot delegates to the planner (Facade). PlotPlanner (src/main/java/ca/yorku/storyteller/agent/PlotPlanner.java): createPlot builds the request from Scenario.describe() and the basis, parses the reply; planStep turns the next pending event into hidden guidance for each turn. Plot, PivotalEvent, EventStatus (src/main/java/ca/yorku/storyteller/model/): the hidden ordered events. Scenario (src/main/java/ca/yorku/storyteller/model/Scenario.java): readyToStart checks mandatory fields. prompts/plot-planner.txt (src/main/resources/prompts/plot-planner.txt): the planning instructions. StoryCLI (src/main/java/ca/yorku/storyteller/ui/StoryCLI.java): the start command.
Key Methods	StoryController.startStory(Scenario, String), Scenario.readyToStart(), StoryAgent.planPlot(Scenario, String), PlotPlanner.createPlot(Scenario, String), PlotPlanner.parseEvents(String), PlotPlanner.planStep(StorySession, String), Plot.nextPending().
Design Patterns	Facade: the controller only calls StoryAgent.planPlot. Adapter: the request goes through LLMClient. MVC: the CLI collects the fields and calls the controller; the plan never reaches a view.

Collaboration (Input to Result)	1. The user types start and answers the scenario questions, ending with an optional plot basis. 2. StoryCLI builds a Scenario and calls StoryController.startStory(scenario, basis). 3. The controller calls scenario.readyToStart() (premise and setting are required; a title is made if missing). 4. The controller calls StoryAgent.planPlot, which calls PlotPlanner.createPlot. 5. The planner sends the scenario and the basis with the planning prompt through LLMClient.generate and parses {"events": [...]} from the reply into PivotalEvents E1..En, all PENDING. 6. The controller puts the scenario and the Plot on the session (the open empty one, or a new one) and notifies the observers; the CLI prints only that the story has begun. 7. On every later turn PlotPlanner.planStep sends the next pending event to the narrator in a hidden section.
GUI and CLI Access	GUI: the Plot basis field of the Start story form (optional). CLI: the last question of start. Neither front end ever shows the events.
Error / Alternative Behaviour	Empty premise or setting: a message, and no model call. Claude declines, the call fails, or the reply has no events: an error message ("Claude did not return any pivotal events. Try starting the story again.") and no story is started; the open session is unchanged.
Video Link and Timestamp	*[TO ADD: video name, link and mm:ss, matching Section 7]*
Changes from Stage 1	planStep builds the guidance without a second model call; starting keeps lore prepared beforehand (Section 4).

F06: Plot Steering

Feature ID and Name	F06: Plot Steering
Stage 1 Reference	Stage 1 report section 2.2 and section 9 (F06); UC04 Take a story turn; SD05 The agent narrates the turn
Implementation Status	Done. Runs inside every story turn; the user steers by playing, and the events stay hidden in both front ends.
Type	AI, as in Stage 1. The model decides when an event has happened and when the plan should change; the code checks every update_plot call (known event id, 1 to 6 new events, nothing to revise once the plan is complete), numbers new events and applies them to the Plot.

Classes / Files and Responsibilities	PlotPlanner (src/main/java/ca/yorku/storyteller/agent/PlotPlanner.java): planStep lists the events reached, the next pending event and the later ones, and says when to call update_plot. StoryAgent (src/main/java/ca/yorku/storyteller/agent/StoryAgent.java): adds the guidance to the turn prompt and offers update_plot in the tool loop; create registers the tool. ToolRegistry (src/main/java/ca/yorku/storyteller/agent/tools/ToolRegistry.java): checks the required arguments and runs the call. UpdatePlotTool (src/main/java/ca/yorku/storyteller/agent/tools/UpdatePlotTool.java): action "reached" marks an event; action "revise" replaces the pending events. Plot, PivotalEvent (src/main/java/ca/yorku/storyteller/model/): markReached, replacePending, nextPending, isComplete. SubmitTurnCommand (src/main/java/ca/yorku/storyteller/controller/SubmitTurnCommand.java): copies the plot before the turn and puts it back on undo or failure. prompts/system.txt (src/main/resources/prompts/system.txt): tells the narrator to use the guidance quietly and never reveal it.
Key Methods	PlotPlanner.planStep(StorySession, String), StoryAgent.takeTurn(StorySession, String, InputModeStrategy), ToolRegistry.execute(ToolCall, StorySession), UpdatePlotTool.execute(Map, StorySession), UpdatePlotTool.parseEvents(String), Plot.markReached(String), Plot.replacePending(List), Plot.nextPending(), SubmitTurnCommand.undo().
Design Patterns	Facade: the planner and the tool are part of the subsystem behind StoryAgent.takeTurn. Adapter: the model is reached through LLMClient. MVC: the views only send the input; the plan never reaches them. Command: SubmitTurnCommand restores the plot on undo.

Collaboration (Input to Result)	1. The user plays a turn in either front end (Send, or do / say / story <text>). 2. StoryAgent.takeTurn asks PlotPlanner.planStep for the hidden guidance: events reached, the next event, the later events, and when to call update_plot. 3. The guidance goes into the turn prompt with the update_plot tool schema. 4. When the narration reaches the next event, the model calls update_plot with action "reached" and the event id; when the user has gone off-script, it calls action "revise" with new events. 5. ToolRegistry.execute validates the call and UpdatePlotTool.execute changes the Plot; the result (and the new next event) goes back to the model, which finishes the narration. 6. The user sees only the narration; the next turn's guidance reflects the updated plan.
GUI and CLI Access	No control of its own, by design: it runs during every turn from the story view (GUI) and do / say / story (CLI). Neither front end shows the events.
Error / Alternative Behaviour	Unknown event id, unknown action, a revision with no events or more than 6, or a revision once every event is reached: a failed ToolResult goes back to the model and the turn continues. An event reached out of order is accepted and the result names the earlier pending one. Undo, retry or a failed turn puts the plot back as it was.
Video Link and Timestamp	*[TO ADD: video name, link and mm:ss, matching Section 7]*
Changes from Stage 1	One update_plot tool with an action argument; the guidance lists every pending event (Section 4, "F06: UpdatePlotTool takes an action").

F07: Lore Proposals

Feature ID and Name	F07: Lore Proposals
Stage 1 Reference	Stage 1 report section 2.2 and section 9 (F07); UC05 Review lore proposals; SD07 Propose and review lore
Implementation Status	Done. Reachable from the GUI (Proposals tab of the sidebar) and the CLI (proposals, approve, reject).
Type	Hybrid, as in Stage 1. The model decides what to propose (propose_lore tool call); validating the proposal, queuing it as pending and applying the user's decision are deterministic code.

Classes / Files and Responsibilities	ProposeLoreTool (src/main/java/ca/yorku/storyteller/agent/tools/ProposeLoreTool.java): validates title and content, refuses titles already in the lore book or pending, queues a LoreProposal. ToolRegistry (src/main/java/ca/yorku/storyteller/agent/tools/ToolRegistry.java): checks the required arguments and runs the tool. StoryAgent (src/main/java/ca/yorku/storyteller/agent/StoryAgent.java): registers the tool; the model calls it during a turn. StorySession (src/main/java/ca/yorku/storyteller/model/StorySession.java): addProposal, approveProposal, rejectProposal, getPendingProposals. LoreProposal, ProposalStatus (src/main/java/ca/yorku/storyteller/model/): the proposal and its state. StoryController (src/main/java/ca/yorku/storyteller/controller/StoryController.java): resolveProposal(id, approved), listPendingProposals. LoreBook, LoreEntry (src/main/java/ca/yorku/storyteller/model/): receive an approved proposal. StoryGUI (src/main/java/ca/yorku/storyteller/ui/StoryGUI.java): proposals list with Approve and Reject and a waiting count. StoryCLI (src/main/java/ca/yorku/storyteller/ui/StoryCLI.java): proposals, approve <id>, reject <id>; the status line counts waiting proposals.
Key Methods	ProposeLoreTool.execute(Map, StorySession), ToolRegistry.execute(ToolCall, StorySession), StorySession.addProposal(LoreProposal), StoryController.listPendingProposals(), StoryController.resolveProposal(String, boolean), StorySession.approveProposal(String), LoreProposal.approve(), StorySession.rejectProposal(String), LoreBook.add(LoreEntry), StoryGUI.resolveSelected(boolean).
Design Patterns	Facade and Adapter: the proposal comes from the model inside StoryAgent's tool loop. MVC: only StoryController.resolveProposal moves a proposal into the lore book. Observer: the views redraw the proposals list and the count on every notification.

Collaboration (Input to Result)	1. During a turn the model calls propose_lore with a title and content. 2. ToolRegistry.execute checks both are present and runs ProposeLoreTool.execute, which refuses a duplicate title or creates a LoreProposal (PENDING) and calls StorySession.addProposal; the lore book is not touched. 3. When the turn ends the session notifies the observers; the GUI shows the proposal in the sidebar's Proposals tab with the count in the tab title, the status bar and, while the sidebar is collapsed, its button (CLI: the status line says how many are waiting). 4. The user selects it and clicks Approve or Reject (CLI: approve <id> / reject <id>). 5. The view calls StoryController.resolveProposal(id, approved): approve calls StorySession.approveProposal, which calls LoreProposal.approve() and LoreBook.add(entry); reject calls rejectProposal. 6. The controller notifies the observers and the lists refresh.
GUI and CLI Access	GUI: Proposals tab of the sidebar (list, the proposed text, Approve and Reject); the tab title shows "Proposals (N)". CLI: proposals, approve <id>, reject <id>.
Error / Alternative Behaviour	Empty title or content, or a title already used: the tool call fails, the model is told why and nothing is shown to the user. Unanswered proposals stay pending. Approving a proposal whose title was taken meanwhile: an error and it stays pending. Unknown or already resolved id: "There is no pending proposal P2." Proposals raised during a turn that is undone, or that fails afterwards, stay in the list.
Video Link and Timestamp	*[TO ADD: video name, link and mm:ss, matching Section 7]*
Changes from Stage 1	Duplicate-title check, listPendingProposals, and the list redrawn on notify instead of showProposal (Section 4).

F08: Consistency Check

Feature ID and Name	F08: Consistency Check
Stage 1 Reference	Stage 1 report section 2.2 and section 9 (F08); UC04 Take a story turn; SD06 Check consistency and update memory
Implementation Status	Done. Warnings appear under the turn in both front ends; they never block it.
Type	Hybrid, as in Stage 1. The code picks the lore to check (pinned entries and the entries the turn used), skips the check when there are none, parses the reply and formats each warning; the model judges whether the passage contradicts an entry.

Classes / Files and Responsibilities	ConsistencyChecker (src/main/java/ca/yorku/storyteller/agent/ConsistencyChecker.java): sends the lore and the passage, and turns the JSON answer into warnings (at most 5). StoryAgent (src/main/java/ca/yorku/storyteller/agent/StoryAgent.java): takeTurn checks the narration with the lore ids collected in the tool loop. LoreBook, LoreEntry (src/main/java/ca/yorku/storyteller/model/): getPinned and find supply the entries. Turn (src/main/java/ca/yorku/storyteller/model/Turn.java): holds the warnings; setWarnings is used by edits. EditTurnCommand (src/main/java/ca/yorku/storyteller/controller/EditTurnCommand.java): clears the warnings on an edit and restores them on undo. StoryGUI, StoryCLI (src/main/java/ca/yorku/storyteller/ui/): show each warning under its turn. prompts/consistency-check.txt (src/main/resources/prompts/consistency-check.txt): the checker's instructions and answer format.
Key Methods	StoryAgent.takeTurn(StorySession, String, InputModeStrategy), ConsistencyChecker.check(String, StorySession, Collection<String>), ConsistencyChecker.parseWarnings(String, Map), LoreBook.getPinned(), Turn.getWarnings(), Turn.setWarnings(List), EditTurnCommand.execute(), EditTurnCommand.undo().
Design Patterns	Facade: ConsistencyChecker is a subsystem class behind StoryAgent.takeTurn. Adapter: the check goes through LLMClient. Observer and MVC: the views redraw the warnings from the turns when notified. Command: EditTurnCommand saves and restores the warnings.
Collaboration (Input to Result)	1. The user plays a turn; the tool loop collects the ids of the lore entries the model looked up. 2. With the narration ready, StoryAgent.takeTurn calls ConsistencyChecker.check(narration, session, usedLoreIds). 3. The checker gathers the pinned entries and the used entries; with none, it returns no warnings without a model call. 4. Otherwise it sends the entries (marked [canon] when pinned) and the passage; the model answers {"contradictions": [{"entry", "problem"}]}. 5. Each item becomes a warning such as Contradicts L1 "Orrin" (canon): ..., stored in the Turn. 6. The turn is added and the views show the narration with the warnings beneath it.
GUI and CLI Access	GUI: the transcript, [Warning: ...] lines under the turn. CLI: Warning: ... lines after the narration and in turns.

Error / Alternative Behaviour	A failed call, a refusal or an unreadable answer adds one warning, "This turn was not checked against the lore", and the turn still goes ahead. An answer naming an unknown entry is ignored. The user can retry or edit the turn (F10); an edit clears its warnings and undo brings them back.
Video Link and Timestamp	*[TO ADD: video name, link and mm:ss, matching Section 7]*
Changes from Stage 1	check takes the lore ids the turn used; an edit clears the warnings (Section 4, "F08: The check takes the lore the turn used").

F09: Story Memory Summarization

Feature ID and Name	F09: Story Memory Summarization
Stage 1 Reference	Stage 1 report section 2.2 and section 9 (F09); UC04 Take a story turn; UC07 Save and load a story; SD06 Check consistency and update memory; SD09 Save and load a story
Implementation Status	Done. Runs automatically during turns; its result is visible as the recap shown when a story is loaded (F11).
Type	AI, as in Stage 1. The code decides when to summarize (more turns than the window of 8), which turns to add, and what to send; the model writes the summary and the recap.
Classes / Files and Responsibilities	MemoryManager (src/main/java/ca/yorku/storyteller/agent/MemoryManager.java): summarizeIfNeeded folds the turns that left the window into the summary; buildContext sends the summary and the uncovered turns; recap writes the welcome-back text. StoryAgent (src/main/java/ca/yorku/storyteller/agent/StoryAgent.java): calls summarizeIfNeeded at the start of each turn; recap(session) for the controller. StorySession (src/main/java/ca/yorku/storyteller/model/StorySession.java): summary and summarizedUpTo, saved with the story. StoryController (src/main/java/ca/yorku/storyteller/controller/StoryController.java): loadSession asks for the recap. prompts/summarizer.txt, prompts/recap.txt (src/main/resources/prompts/): the instructions for each.
Key Methods	MemoryManager.summarizeIfNeeded(StorySession), MemoryManager.buildContext(StorySession), MemoryManager.recap(StorySession), StoryAgent.takeTurn(StorySession, String, InputModeStrategy), StoryAgent.recap(StorySession), StorySession.setSummary(String, int), StoryController.loadSession(String).

Design Patterns	Facade: MemoryManager sits behind StoryAgent (takeTurn and recap). Adapter: both calls go through LLMClient. MVC: the recap reaches the views as the controller's return value.
Collaboration (Input to Result)	1. The user plays a turn; StoryAgent.takeTurn first calls MemoryManager.summarizeIfNeeded(session). 2. When there are more than 8 turns not yet in the summary window, the manager sends the summary so far and the turns that have left the window, and stores the reply with StorySession.setSummary(text, upTo). 3. buildContext then gives the turn prompt the summary followed by the most recent turns. 4. When a saved story is loaded, StoryController.loadSession calls StoryAgent.recap, which asks MemoryManager.recap for a short recap from the scenario and the context (never the hidden plan). 5. The GUI shows the recap in the "Story recap" card above the transcript; the CLI prints it after the load line.
GUI and CLI Access	Summarizing has no control of its own; it runs during turns. The recap: GUI Story > Load... (a "Story recap" card above the transcript, with Dismiss); CLI load <id>.
Error / Alternative Behaviour	A failed or refused summary keeps the old summary and is retried on the next turn; meanwhile buildContext also sends the uncovered older turns (up to 16), so nothing is lost. A failed recap still loads the story and the message ends with "(No recap this time: <reason>)". A story without turns gets no recap and no model call.
Video Link and Timestamp	*[TO ADD: video name, link and mm:ss, matching Section 7]*
Changes from Stage 1	Summarizing happens at the start of a turn; the recap is shown with the load message (Section 4, "F09: Older turns are summarized" and "F09/F11: The recap").

F10: Undo / Retry / Edit Turn

Feature ID and Name	F10: Undo / Retry / Edit Turn
Stage 1 Reference	Stage 1 report section 2.2 and section 9 (F10); UC06 Undo, retry or edit a turn; SD08 Undo, retry or edit a turn
Implementation Status	Done. Reachable from the GUI (Undo, Retry, Edit buttons) and the CLI (undo, retry, edit).
Type	Hybrid, as in Stage 1: undo and edit are deterministic; retry calls the model again through a new SubmitTurnCommand.

Classes / Files and Responsibilities	StoryGUI (src/main/java/ca/yorku/storyteller/ui/StoryGUI.java): Undo, Retry and Edit buttons, enabled only when they apply; Edit opens an editable box with OK and Cancel; a failed retry puts the input and mode back. StoryCLI (src/main/java/ca/yorku/storyteller/ui/StoryCLI.java): undo, retry, edit (multi-line, ends with a dot). StoryController (src/main/java/ca/yorku/storyteller/controller/StoryController.java): undoTurn, retryTurn, editTurn. CommandHistory (src/main/java/ca/yorku/storyteller/controller/CommandHistory.java): run, undoLast, canUndo, latest, peekLast, clear (on new, start and load). SubmitTurnCommand (src/main/java/ca/yorku/storyteller/controller/SubmitTurnCommand.java): undo removes the turn and restores plotBefore; getInput and getMode for retry. EditTurnCommand (src/main/java/ca/yorku/storyteller/controller/EditTurnCommand.java): swaps the narration and restores oldNarration. StorySession, Turn, Plot (src/main/java/ca/yorku/storyteller/model/): removeLastTurn, lastTurn, setNarration, copy.
Key Methods	StoryController.undoTurn(), StoryController.retryTurn(), StoryController.editTurn(String), CommandHistory.canUndo(), CommandHistory.undoLast(), CommandHistory.latest(Class), CommandHistory.run(StoryCommand), SubmitTurnCommand.undo(), SubmitTurnCommand.getInput(), SubmitTurnCommand.getMode(), EditTurnCommand.execute(), EditTurnCommand.undo(), StorySession.removeLastTurn(), StorySession.lastTurn(), Plot.copy().
Design Patterns	Command (as in Stage 1): each action is a StoryCommand that knows how to undo itself; CommandHistory is the invoker. Strategy: retry reuses the original InputModeStrategy. MVC and Observer: the views forward the button press and redraw on notify.
Collaboration (Input to Result)	1. Undo: the user clicks Undo (CLI: undo); StoryController.undoTurn calls CommandHistory.undoLast. For a SubmitTurnCommand that removes the turn and restores the plot copy taken before it; for an EditTurnCommand it restores the original narration. 2. Retry: StoryController.retryTurn finds the latest SubmitTurnCommand with CommandHistory.latest, undoes any edits above it and the turn itself, then runs a new SubmitTurnCommand with getInput() and getMode(), which narrates the turn again as in F04. 3. Edit: the user clicks Edit, changes the text and clicks OK (CLI: edit, then the text and a line with a dot); StoryController.editTurn rejects empty text and runs an EditTurnCommand on StorySession.lastTurn(). 4. After each action the controller notifies the observers and the transcript and buttons refresh.

GUI and CLI Access	GUI: Undo, Retry and Edit under the transcript. CLI: undo, retry, edit.
Error / Alternative Behaviour	Nothing to undo or retry (including right after a load): the buttons are disabled (CLI: "There is nothing to undo." / "There is no turn to retry."). Retry fails: the turn stays undone, the error is shown and the input and mode are put back in the box (CLI: the input is printed). Edit left empty: "The narration cannot be empty..." and nothing changes. Edit cancelled: nothing changes. Undo after an edit: the original narration comes back and the turn stays.
Video Link and Timestamp	*[TO ADD: video name, link and mm:ss, matching Section 7]*
Changes from Stage 1	Retry also undoes edits of the turn; SubmitTurnCommand restores the plot when the agent fails; edit works after a load (Section 4).

F11: Save and Load Session

Feature ID and Name	F11: Save and Load Session
Stage 1 Reference	Stage 1 report section 2.2 and section 9 (F11); UC07 Save and load a story; SD09 Save and load a story
Implementation Status	Done for save, list and load from both front ends, with a model-written recap after a load (F09).
Type	Deterministic, as in Stage 1. Saving and loading never call the model; only the recap after a load does, and a failed recap never stops the load.

Classes / Files and Responsibilities	StoryGUI (src/main/java/ca/yorku/storyteller/ui/StoryGUI.java): Story menu with New, Save and Load… (a list of saved stories, newest first). StoryCLI (src/main/java/ca/yorku/storyteller/ui/StoryCLI.java): new, save, load <id>, sessions. StoryController (src/main/java/ca/yorku/storyteller/controller/StoryController.java): saves the current session; loads one, moves the observers to it and clears the undo history. SessionRepository (src/main/java/ca/yorku/storyteller/persistence/SessionRepository.java): the save / load / list interface. JsonSessionRepository (src/main/java/ca/yorku/storyteller/persistence/JsonSessionRepository.java): one JSON file per story in the save directory (STORYTELLER_SAVE_DIR, default saves/), written atomically. CommandHistory (src/main/java/ca/yorku/storyteller/controller/CommandHistory.java): cleared on new and load so undo never reaches back past a load. StoryAgent, MemoryManager (src/main/java/ca/yorku/storyteller/agent/): write the recap (F09). StoryGUI also opens to a start screen (refreshHome): "Select story" with the saved stories by title, or "No stories detected" with Create story…. StoryCLI opens to the same list (printStartScreen). StoryController.listSavedStories() maps each saved id to StorySession.getTitle().
Key Methods	StoryController.saveSession(), StoryController.loadSession(String), StoryController.listSessions(), StoryController.newSession(), SessionRepository.save(StorySession), SessionRepository.load(String), SessionRepository.listSessions(), StorySession.attach(StoryObserver), CommandHistory.clear(), StoryAgent.recap(StorySession), StoryGUI.showRecap(String), StoryController.listSavedStories(), StorySession.getTitle(), StoryGUI.refreshHome(), StoryGUI.openSelectedStory(), SessionRepository.lastSaved(String).
Design Patterns	MVC and Observer: the menu and commands forward to StoryController, and after a load the controller attaches the front ends to the loaded session and notifies them. Facade: the recap comes from StoryAgent.recap.

Collaboration (Input to Result)	1. Save: Story > Save or save calls StoryController.saveSession(), which calls SessionRepository.save(session); JsonSessionRepository writes scenario, plot, lore, characters, turns, proposals and summary to <id>.json through a temporary file. 2. Load: Story > Load... or sessions lists SessionRepository.listSessions(); choosing a story (or load <id>) calls StoryController.loadSession(id). 3. The controller calls SessionRepository.load(id), detaches the front ends from the old session, makes the loaded one current, clears CommandHistory, attaches the front ends and calls notifyObservers(). 4. If the story has turns, the controller asks StoryAgent.recap(session) for a recap and returns it after the "Loaded ..." line; the GUI shows it in the "Story recap" card above the transcript and the CLI prints it.
GUI and CLI Access	GUI: the start screen ("Select story", each story with its save time in grey, with Open and Create story..., or "No stories detected" with Create story...), then the Story menu, New / Save / Load... (stories listed by title). CLI: the start screen lists saved ids, titles and save times; new, save, sessions, load <id>.
Error / Alternative Behaviour	A missing save, a damaged file or a file whose id does not match its name produces a message and the current story stays open. A failed write leaves no partial file and the story stays open. Saving with no story open is refused.
Video Link and Timestamp	*[TO ADD: video name, link and mm:ss, matching Section 7]*
Changes from Stage 1	Save errors are exceptions from the repository; the recap follows a "Loaded ..." line and a failed recap is explained (Section 4, "F11: Save errors are exceptions" and "F09/F11: The recap"). The app opens to a story picker and lists stories by title (Section 4, "F11: The app opens to a story picker").

F12: Ending Generation

Feature ID and Name	F12: Ending Generation
Stage 1 Reference	Stage 1 report section 2.2 and section 9 (F12); UC08 End the story; SD10 End the story
Implementation Status	Done. End story in the GUI and end in the CLI; both offer to end once the last pivotal event is reached.
Type	AI, as in Stage 1. The code decides when an ending is possible (a started story with a turn), when to suggest it (the turn that completed the plan) and when to ask for confirmation (events still pending), and stores the result; the model writes the closing passage.

Classes / Files and Responsibilities	StoryGUI (src/main/java/ca/yorku/storyteller/ui/StoryGUI.java): End story button ("Rewrite ending" once ended), confirmation dialogs, the ending at the bottom of the transcript. StoryCLI (src/main/java/ca/yorku/storyteller/ui/StoryCLI.java): end command, the offer after a turn, prints the ending ending with THE END. StoryController (src/main/java/ca/yorku/storyteller/controller/StoryController.java): requestEnding, canEnd, isReadyToEnd, isEndingSuggested, isEnded; blocks turns, undo, retry and edit once ended. StoryAgent (src/main/java/ca/yorku/storyteller/agent/StoryAgent.java): readyToEnd and generateEnding. PlotPlanner (src/main/java/ca/yorku/storyteller/agent/PlotPlanner.java): readyToEnd(plot) is Plot.isComplete(). MemoryManager (src/main/java/ca/yorku/storyteller/agent/MemoryManager.java): buildContext supplies the story so far. StorySession (src/main/java/ca/yorku/storyteller/model/StorySession.java): ending, setEnding, isEnded; saved with the story. prompts/ending.txt (src/main/resources/prompts/ending.txt): the closing-passage instructions.
Key Methods	StoryController.requestEnding(), StoryController.isReadyToEnd(), StoryController.isEndingSuggested(), StoryController.canEnd(), StoryAgent.readyToEnd(StorySession), PlotPlanner.readyToEnd(Plot), StoryAgent.generateEnding(StorySession), MemoryManager.buildContext(StorySession), StorySession.setEnding(String), StoryGUI.endStory(), StoryCLI.end().
Design Patterns	Facade: the controller calls StoryAgent.readyToEnd and generateEnding only. Adapter: the ending is requested through LLMClient. MVC and Observer: the controller stores the ending and notifies, and both views redraw the story with the ending.
Collaboration (Input to Result)	1. After a turn that reaches the last pivotal event, isEndingSuggested() is true and the view asks "The story has reached its final turning point. End it now?" Otherwise the user clicks End story (CLI: end) and, if events are still pending, confirms ending early. 2. The view calls StoryController.requestEnding(). 3. The controller calls StoryAgent.generateEnding(session), which sends the scenario, the plan with each event marked happened or not, whether the plan was completed, every lore entry ([canon] when pinned) and MemoryManager.buildContext(session). 4. The controller stores the reply with StorySession.setEnding and notifies the observers. 5. Both views show the ending after the last turn, followed by THE END; turn controls are disabled.

GUI and CLI Access	GUI: End story button under the transcript, across from Undo, Retry and Edit (Rewrite ending once the story has ended); a dialog offers the ending when the plan completes. CLI: end; the offer appears after the turn or retry that completes the plan.
Error / Alternative Behaviour	No turn yet: "Take at least one turn before ending the story." Events pending: the user confirms or cancels. Claude declines, the call fails or the reply is empty: a message and the story is unchanged. After an ending, turns, undo, retry and edit are refused with a message; End story asks before writing a new ending in place of the old one.
Video Link and Timestamp	*[TO ADD: video name, link and mm:ss, matching Section 7]*
Changes from Stage 1	The ending is stored on the session; it is suggested once, and an ending closes the story (Section 4, "F12: The ending is stored" and "F12: How the ending is suggested").

3. Design Patterns

Stage 1 designed six patterns (Adapter, Command, Facade, MVC, Observer, Strategy); at least five are required. For each pattern as implemented: the problem it solves, its participants and their roles, the files and methods where it appears, why it fits, and whether it matches the Stage 1 design.

3.1 Adapter

Pattern	Adapter
Problem Solved	The agent needs a language model, but the Anthropic SDK's request and response types are large, provider-specific and change between SDK versions. Without an adapter every agent class would build SDK objects itself, tests would need a network and an API key, and swapping the model or provider would touch the whole agent package.
Participants and Roles	Target: LLMClient (generate(request)). Adapter: ClaudeAdapter. Adaptee: the anthropic-java SDK's AnthropicClient (client.messages().create(params)). Client: the agent classes (StoryAgent and its subsystems, from week 2). Second Target implementation: FakeLLMClient, a scripted client used by the tests. Provider-neutral data: LLMRequest, LLMResponse, LLMMessage, LLMToolCall, LLMToolResult, LLMToolSpec, LLMException.

Files and Methods	src/main/java/ca/yorku/storyteller/llm/LLMClient.java: LLMResponse generate(LLMRequest request). src/main/java/ca/yorku/storyteller/llm/ClaudeAdapter.java: generate(LLMRequest) calls client.messages().create(toApiBody(request)) and returns fromApiBody(message); MessageCreateParams toApiBody(LLMRequest); LLMResponse fromApiBody(Message); SDK exceptions are mapped to LLMException with a readable message. src/main/java/ca/yorku/storyteller/llm/FakeLLMClient.java: enqueue(LLMResponse), enqueueFailure(LLMException), generate(LLMRequest), getRequests().
Why It Fits	The agent's need is one operation (generate a reply to a conversation with tools), while the SDK offers a wide, typed API. The adapter converts between the two in one class: messages with tool calls and tool results, tool schemas, stop reasons, refusals and token usage. Everything else depends only on LLMClient, so tests run against FakeLLMClient with no paid calls, and ClaudeAdapterTest checks the real adapter against a local HTTP server that plays the Messages API. Without the pattern, SDK types would leak into StoryAgent, PlotPlanner, MemoryManager and ConsistencyChecker.
Match with Stage 1	Changed in detail (see Section 4): the roles are as designed, but LLMRequest carries structured LLMMessage and LLMToolSpec objects instead of strings, toApiBody/fromApiBody convert to and from the SDK's typed objects instead of JSON strings, and the adaptee is the SDK client rather than a raw createMessage(jsonBody) call.
Related Features	*F01, F04, F05, F06, F07, F08, F09, F12*

3.2 Command

Pattern	Command
Problem Solved	The most recent turn must be undoable, retryable and editable, and undoing a turn must also put the hidden pivotal events back as they were before it, even though the turn may have changed them through tool calls.
Participants and Roles	Command: StoryCommand (execute, undo). Concrete commands: SubmitTurnCommand (keeps input, mode, plotBefore and the resulting Turn), EditTurnCommand (keeps the turn and oldNarration). Invoker: CommandHistory (run, undoLast, canUndo, latest, peekLast, clear). Client: StoryController (submitTurn, undoTurn, retryTurn, editTurn). Receivers: StorySession, Turn, and StoryAgent (called by SubmitTurnCommand.execute).

Files and Methods	src/main/java/ca/yorku/storyteller/controller/StoryCommand.java: execute(), undo(). SubmitTurnCommand.java: execute() copies the plot and calls StoryAgent.takeTurn, undo() calls StorySession.removeLastTurn() and setPlot(plotBefore), getInput(), getMode(). EditTurnCommand.java: execute() swaps the narration, undo() restores it. CommandHistory.java: run(command), undoLast(), latest(type). StoryController.java: submitTurn, undoTurn, retryTurn, editTurn.
Why It Fits	Each action knows exactly what it changed, so undo needs no knowledge of what kind of action came last, and retry is just "undo, then run a new command with the same input and mode". Without it, the controller would need its own records of every turn's plot and narration and separate undo code for each action. The history is not saved, so a load starts with nothing to undo.
Match with Stage 1	Matches the Stage 1 roles. Additions: getMode()/getResult(), latest/peekLast on the history, retry undoes edits of the turn first, and edit works after a load (Section 4).
Related Features	*F04, F10*

3.3 Facade

Pattern	Facade
Problem Solved	One story turn involves the input mode, the story context, the hidden plot guidance, several validated tool calls and the model, in a fixed order; starting a story needs a draft and a plan. The controller should ask for "a turn" or "a plan" without knowing that order.
Participants and Roles	Facade: StoryAgent (generateScenario, planPlot, takeTurn, readyToEnd, recap, generateEnding). Subsystem: PlotPlanner (createPlot, planStep, readyToEnd), MemoryManager (buildContext, summarizeIfNeeded, recap), ConsistencyChecker (check), ToolRegistry and its tools, LLMClient. Clients: StoryController (generateScenario, startStory, requestEnding, loadSession, the ending queries) and SubmitTurnCommand (execute).

Files and Methods	src/main/java/ca/yorku/storyteller/agent/StoryAgent.java: create(LLMClient) wires the subsystem; generateScenario(String); planPlot(Scenario, String); takeTurn(StorySession, String, InputModeStrategy); readyToEnd(StorySession); recap(StorySession); generateEnding(StorySession); runToolLoop(...) and buildTurnPrompt(...) keep the step order in one place. src/main/java/ca/yorku/storyteller/agent/PlotPlanner.java, MemoryManager.java, ConsistencyChecker.java, tools/ToolRegistry.java: the subsystem classes. src/main/java/ca/yorku/storyteller/controller/SubmitTurnCommand.java and StoryController.java: the only callers.
Why It Fits	The controller depends on one class; a turn now summarizes, builds context, plans, runs the tool loop and checks consistency, and none of that order is visible to the controller. Adding the consistency check and the summary in week 3 changed StoryAgent and its subsystem only. Without it, StoryController and SubmitTurnCommand would each call the planner, memory, checker, registry and model in the right order and handle the tool loop themselves.
Match with Stage 1	Matches the Stage 1 roles. StoryAgent also offers readyToEnd so the controller does not reach PlotPlanner directly; planStep makes no model call of its own (Section 4).
Related Features	*F01, F04, F05, F06, F07, F08, F09, F11, F12*

3.4 MVC

Pattern	MVC
Problem Solved	The same story must be editable from a Swing GUI and a CLI. Without a separation, each front end would carry its own copy of the validation and storage rules (duplicate names, relationship targets, tag format, save files), and the two would drift apart.
Participants and Roles	Model: StorySession and what it owns (LoreBook, LoreEntry, Character, Relationship, Turn, Plot, LoreProposal), which hold all the rules. View: StoryGUI and StoryCLI, which only collect input, call the controller and display results. Controller: StoryController, the single entry point both views call.

Files and Methods	src/main/java/ca/yorku/storyteller/controller/StoryController.java: saveLoreEntry, removeLoreEntry, searchLore, saveCharacter, removeCharacter, addRelationship, removeRelationship, newSession, saveSession, loadSession, listSessions. src/main/java/ca/yorku/storyteller/ui/StoryGUI.java: saveLoreEntry(), saveCharacter(), addRelationship(), save(), load() each build input and call the controller. src/main/java/ca/yorku/storyteller/ui/StoryCLI.java: handleCommand(String) dispatches each command to the controller. src/main/java/ca/yorku/storyteller/model/: the validation in LoreEntry, LoreBook, StorySession and Character.
Why It Fits	Both views call exactly the same controller methods, so an action behaves the same in both, and the tests drive the controller and the CLI directly (StoryControllerTest, StoryCLITest, StoryGUITest). Without MVC, the CLI and GUI would each need their own copy of the lore and character rules.
Match with Stage 1	Matches the Stage 1 roles. The controller returns errors as exceptions for the view to show instead of calling showError itself, and has extra edit and remove methods (Section 4).
Related Features	*All features (F01 to F12)*

3.5 Observer

Pattern	Observer
Problem Solved	When the story changes (a lore entry is saved, a story is loaded), every open front end must show the new state, but the model should not know whether a GUI, a CLI or both are attached.
Participants and Roles	Subject: StorySession (attach, detach, notifyObservers). Observer: StoryObserver (onStoryChanged). Concrete observers: StoryGUI (refreshes its lists and status bar) and StoryCLI (prints a status line). StoryController registers the front ends with addObserver and moves them to each new or loaded session.
Files and Methods	src/main/java/ca/yorku/storyteller/model/StorySession.java: attach(StoryObserver), detach(StoryObserver), notifyObservers(); the observers field is transient so it is never saved. src/main/java/ca/yorku/storyteller/ui/StoryObserver.java: onStoryChanged(StorySession). src/main/java/ca/yorku/storyteller/ui/StoryGUI.java: onStoryChanged re-reads lore, characters and the status bar on the Swing thread. src/main/java/ca/yorku/storyteller/ui/StoryCLI.java: onStoryChanged prints the counts. src/main/java/ca/yorku/storyteller/controller/StoryController.java: addObserver, and switchTo on new or load.

Why It Fits	The controller notifies once per user action and each view decides how to refresh itself. Without the pattern the controller would have to call refresh methods on specific view classes, coupling it to the GUI and CLI and breaking when only one is open. StorySessionTest checks notification with a recording observer, and a loaded session accepts new observers.
Match with Stage 1	Matches the Stage 1 design, with detach(observer) added so a front end can be moved from the old session to a loaded one.
Related Features	*F02, F03, F04, F07, F08, F10, F11*

3.6 Strategy

Pattern	Strategy
Problem Solved	Do, Say and Story input go through exactly the same turn (context, plot guidance, tool loop, narration); only the way the input is worded for the model differs, and the user picks the mode again on every turn.
Participants and Roles	Strategy: InputModeStrategy (getName, frame). Concrete strategies: DoModeStrategy, SayModeStrategy, StoryModeStrategy. Context: StoryAgent, whose takeTurn(session, input, mode) calls mode.frame(input) and stores mode.getName() on the Turn. Client: StoryController, which picks the strategy from the mode name the user chose.
Files and Methods	src/main/java/ca/yorku/storyteller/agent/InputModeStrategy.java: String getName(), String frame(String input). DoModeStrategy.java, SayModeStrategy.java, StoryModeStrategy.java: frame(input). src/main/java/ca/yorku/storyteller/agent/StoryAgent.java: takeTurn(StorySession, String, InputModeStrategy), buildTurnPrompt(...).
Why It Fits	Each mode is a small object with one varying step, so takeTurn has no branch on the mode name and a new mode is one new class. Without it, StoryAgent would switch on "Do"/"Say"/"Story" wherever the input is worded, and the retry in F10 would have to remember a string and repeat the switch.
Match with Stage 1	Matches the Stage 1 design (getName and frame as in the agent class diagram).
Related Features	*F04, F10*

3.7 Updated UML Class Diagram

Figures 1 to 9 are drawn from the code and saved under figures/stage2/class-diagram/ with the Stage 1 file names (generator: figures/stage2/source/build.py). Figure 1 was redrawn on Nov 14 so that it names every class in the code, including those added in Stage 2 (TranscriptPacer, StoryTheme, SavedStory, LLMMessage, LLMToolCall, LLMToolResult, LLMToolSpec, LLMException). Figures 2 to 8 are as of Nov 3

(end of the F06, F08, F09 and F12 work), with Figures 2, 3 and 4 updated on Nov 9 for the start screen, the gradual text reveal, the sidebar and the palette, and Figure 9 redrawn on Nov 9 for SessionRepository.lastSaved. Every member shown in Figures 2 to 9 was checked against the source on Nov 14.

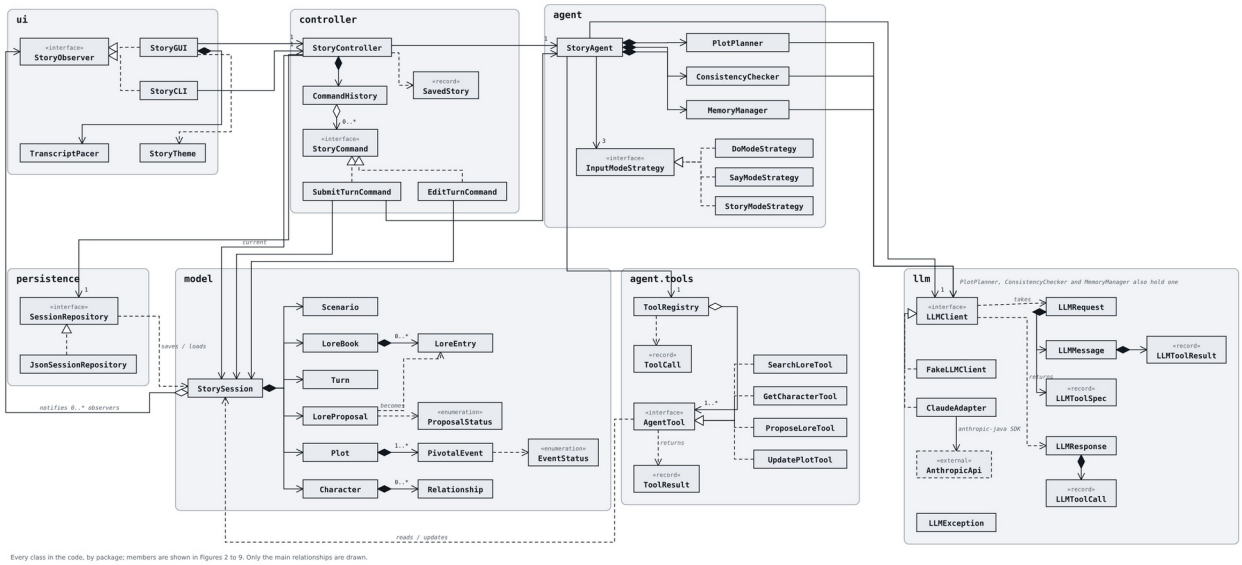


Figure 1. Class diagram overview: every class in the code by package, with the main relationships.

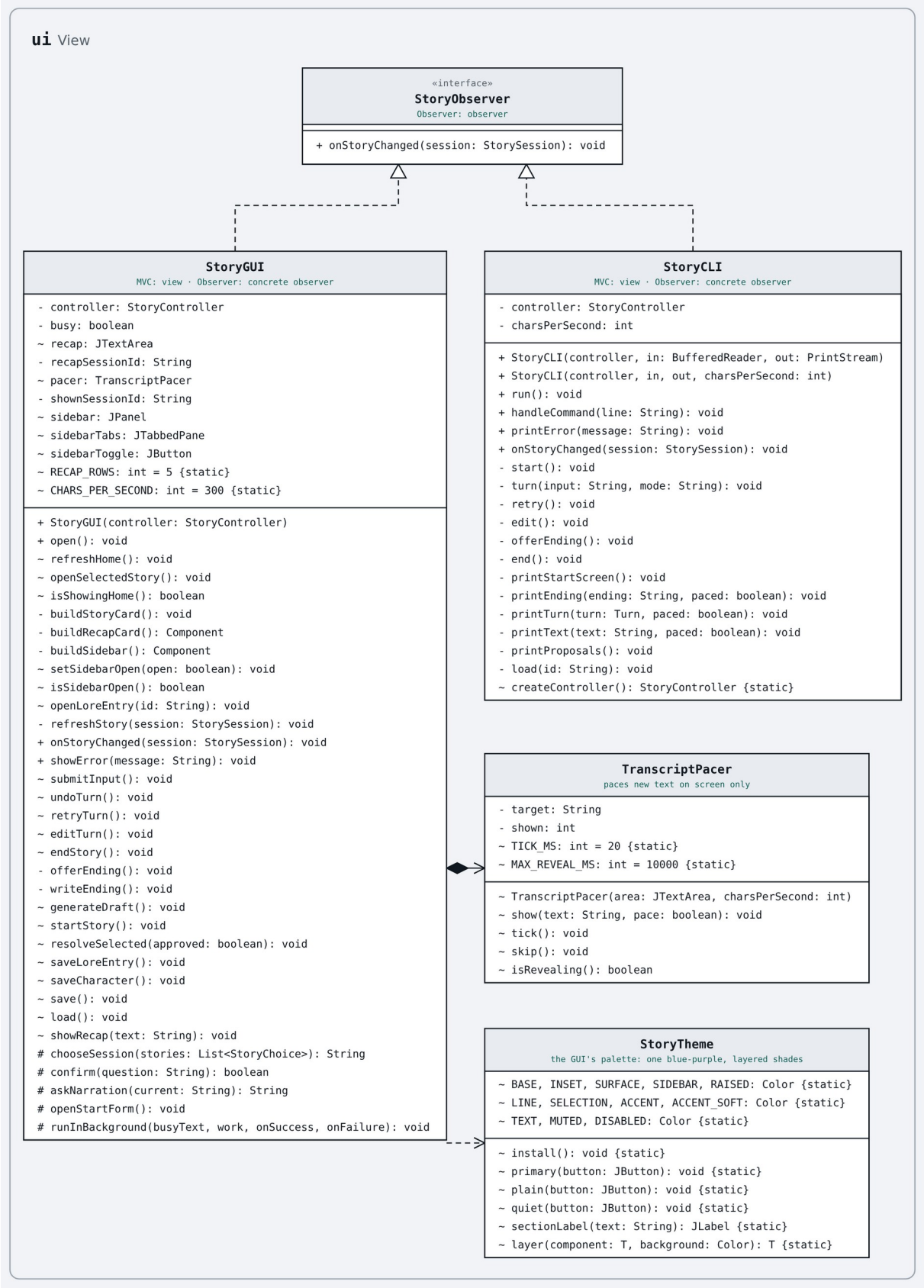


Figure 2. Package ui: the two front ends, the observer interface they implement, and the GUI's pacer and palette.

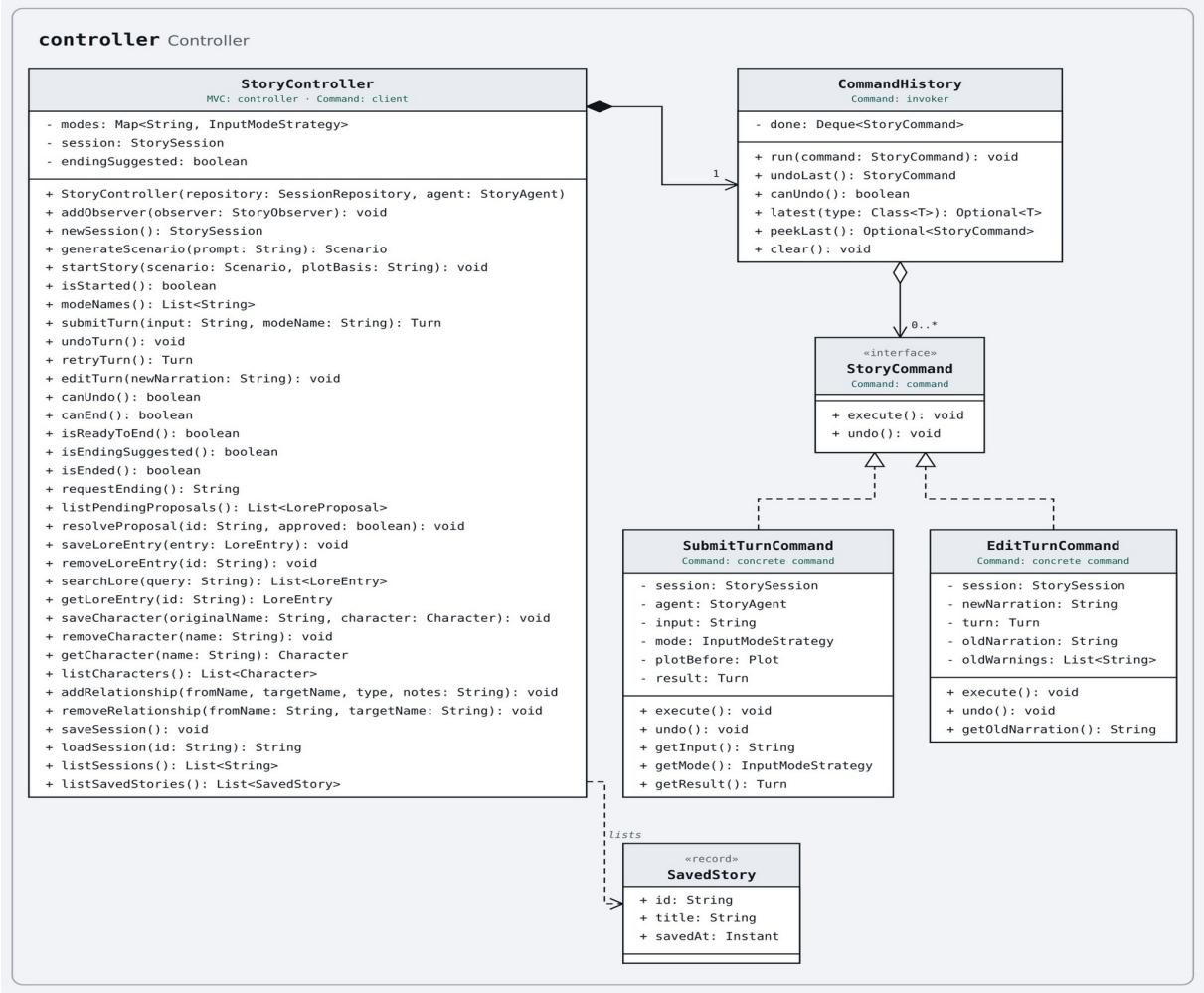


Figure 3. Package controller: the single entry point and the undoable commands.

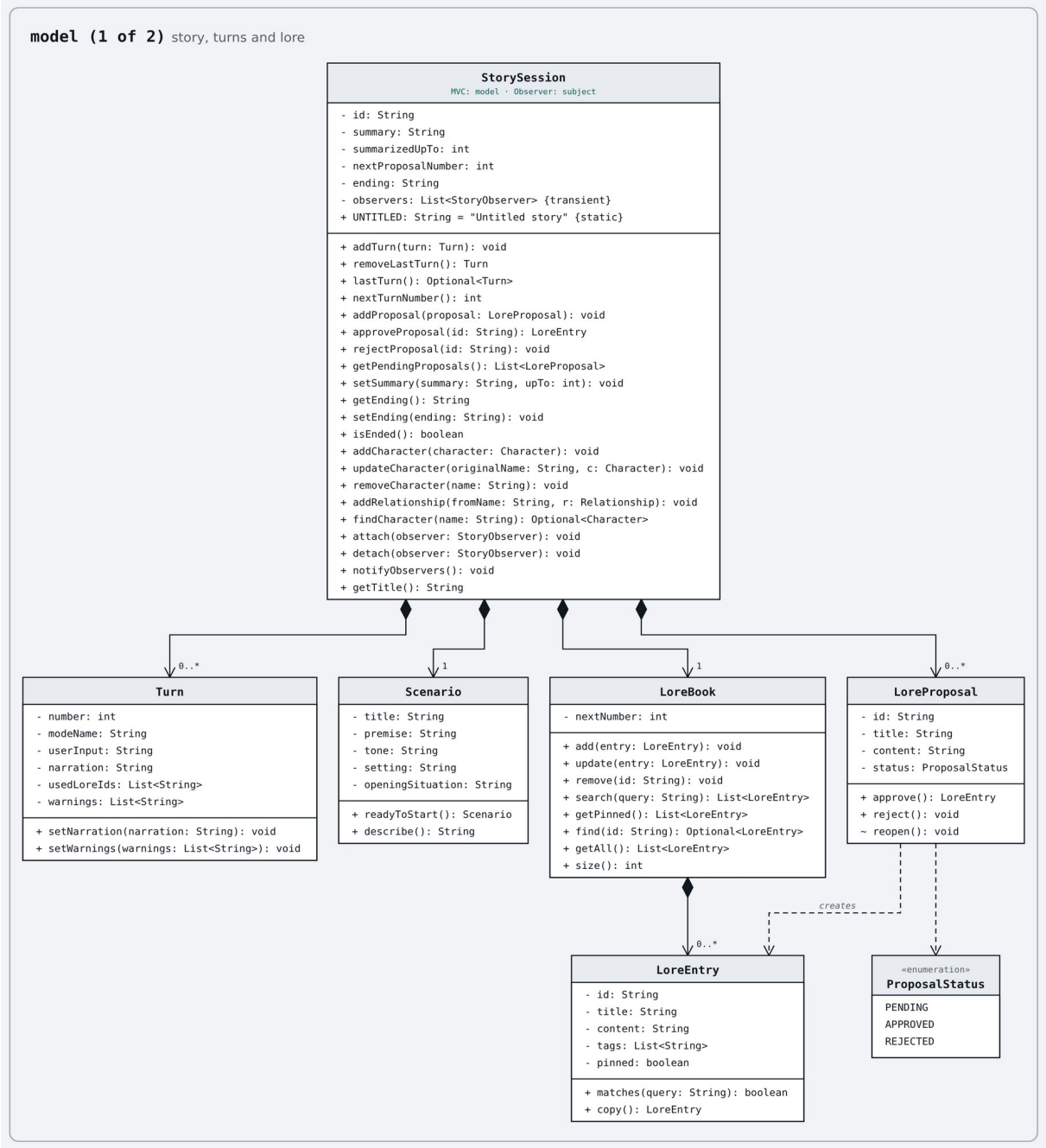


Figure 4. Package model, part 1: the session, its turns, and lore.

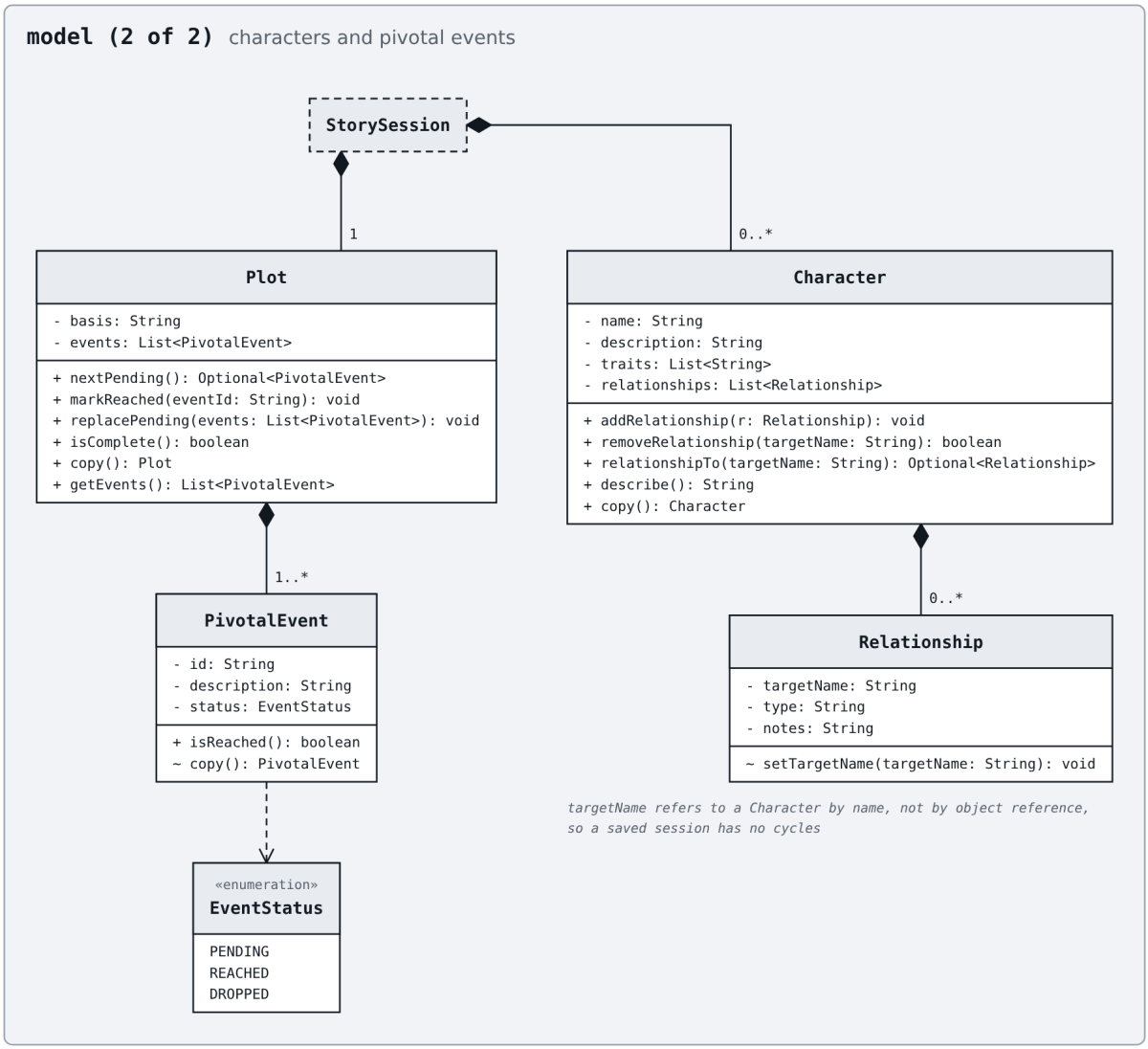


Figure 5. Package model, part 2: characters and the hidden pivotal events.

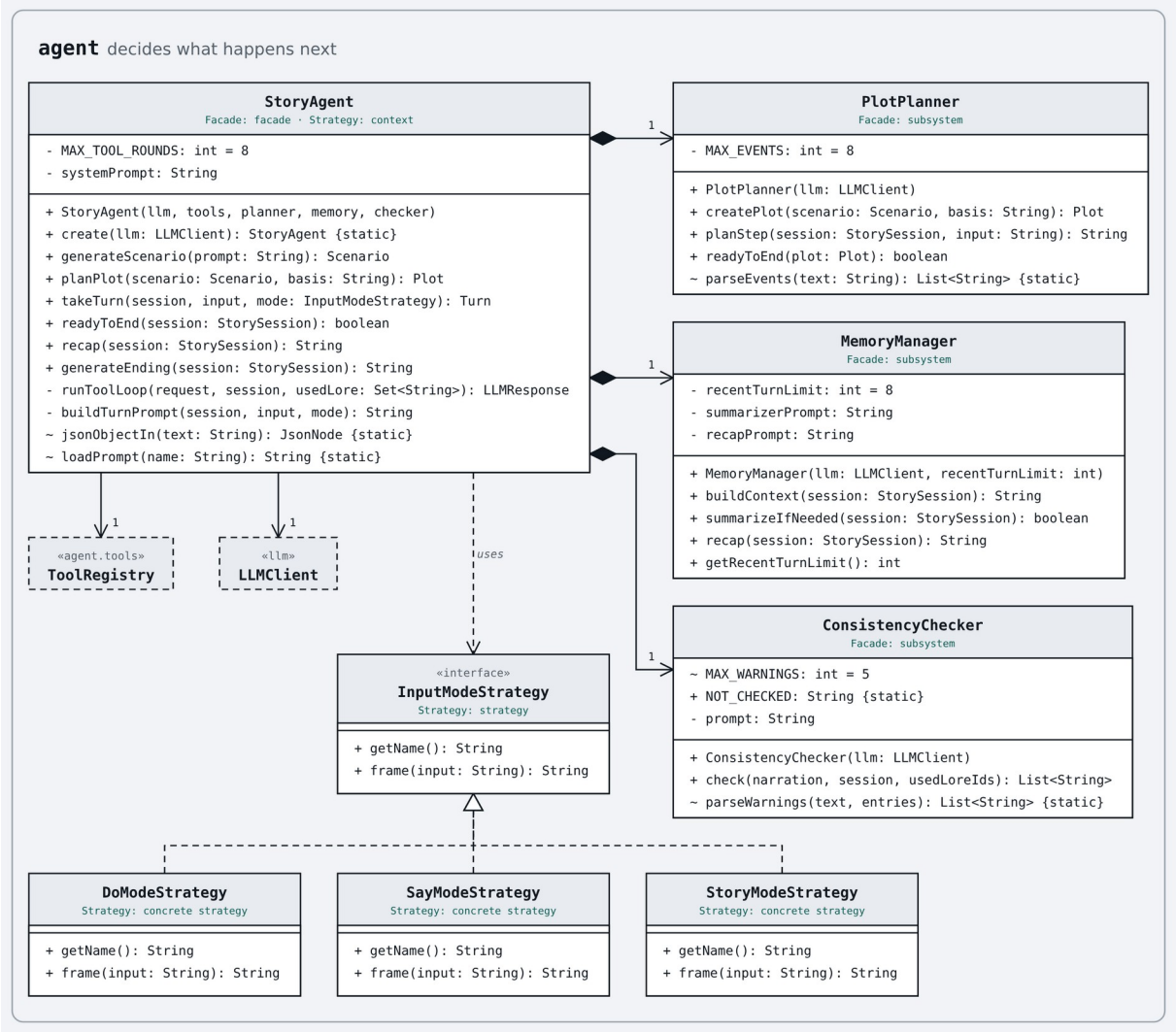


Figure 6. Package agent: the facade, its subsystem classes, and the input-mode strategies.

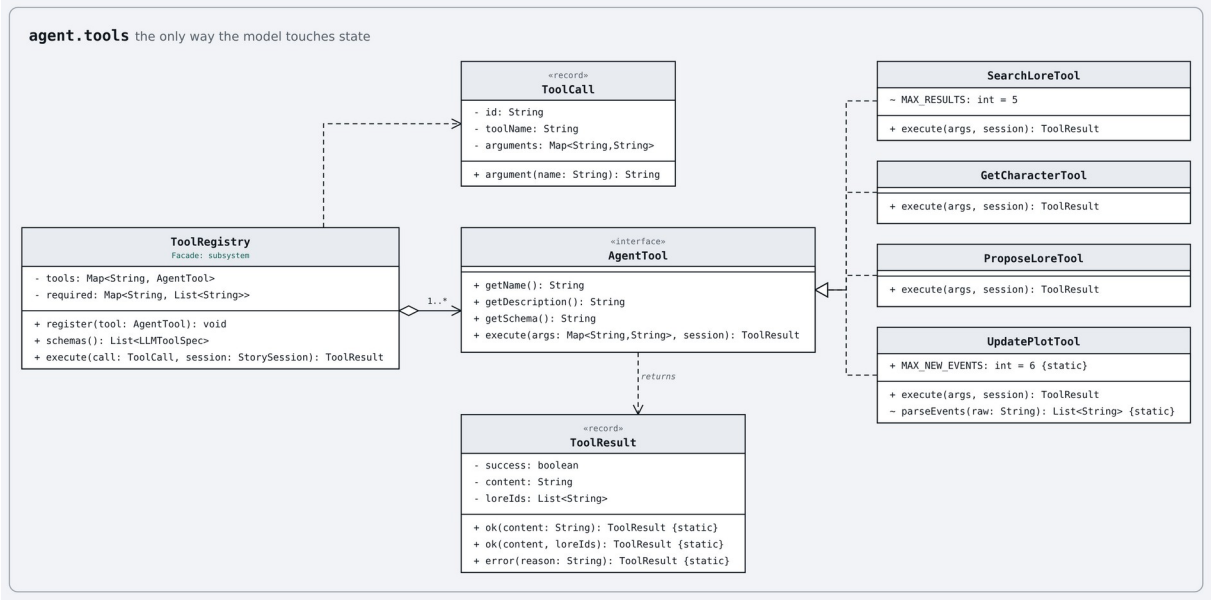


Figure 7. Package agent.tools: the operations the model may request.

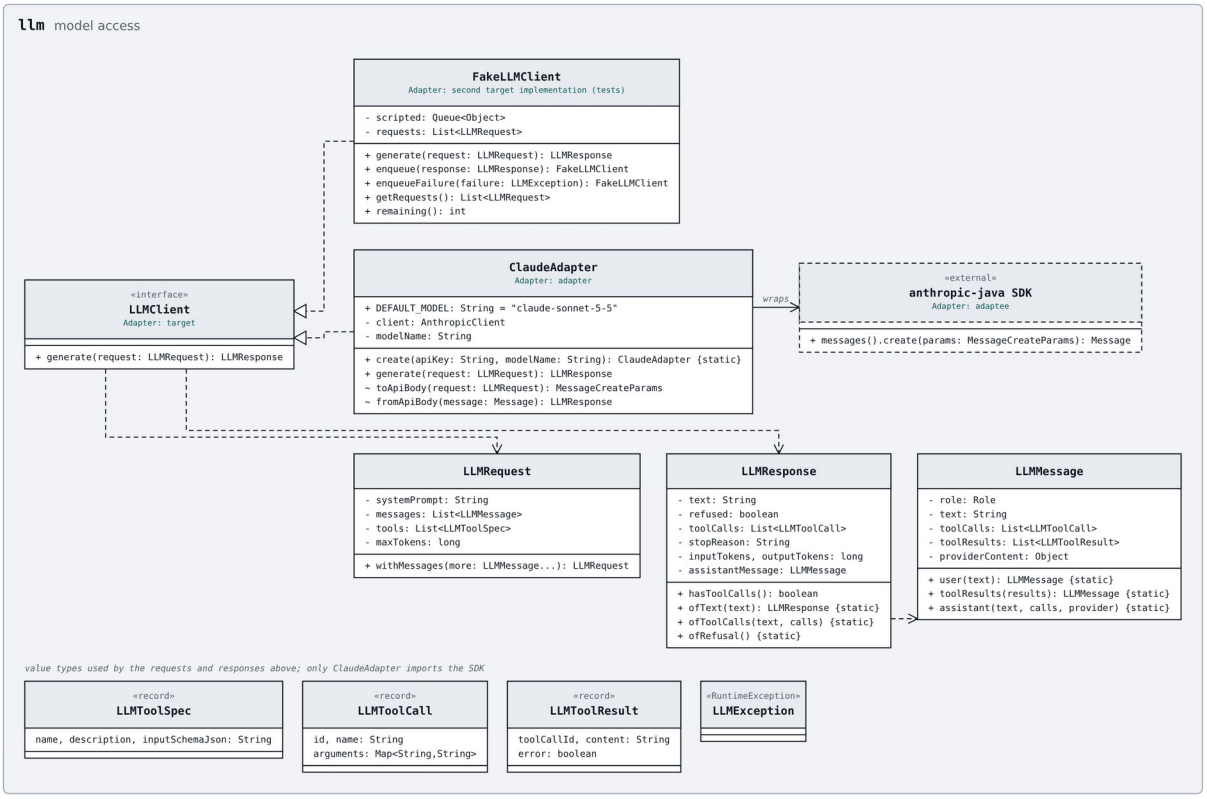


Figure 8. Package llm: the client interface, the adapter for Claude, and the test double.

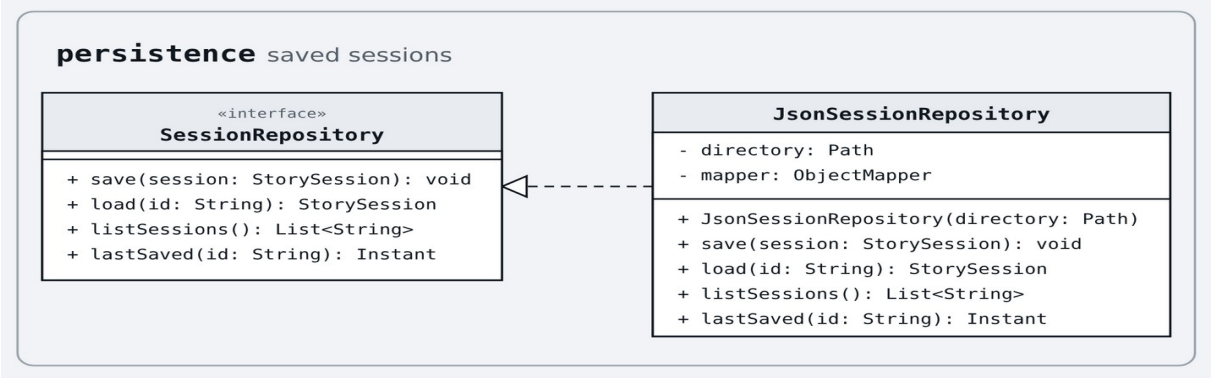


Figure 9. Package persistence: saving and loading sessions.

3.8 Updated Use-Case and Sequence Diagrams

Only diagrams that no longer match the implementation are redrawn; the rest stand as in Stage 1.

Diagram	Still matches?	What changed / new figure
Use-case diagram	Yes	No change; all ten use cases are built.
SD01 Generate a scenario	No	Figure 12.
SD02 Start a story	No	Figure 13.
SD03 Manage lore entries and characters	No	Figure 10.
SD04 Submit a story turn	No	Figure 14.
SD05 The agent narrates the turn	No	Figure 15.
SD06 Check consistency and update memory	No	Figure 16.
SD07 Propose and review lore	No	Figure 17.
SD08 Undo, retry or edit a turn	No	Figure 18.
SD09 Save and load a story	No	Figure 11 (redrawn again in week 3 for the recap).
SD10 End the story	No	Figure 19.

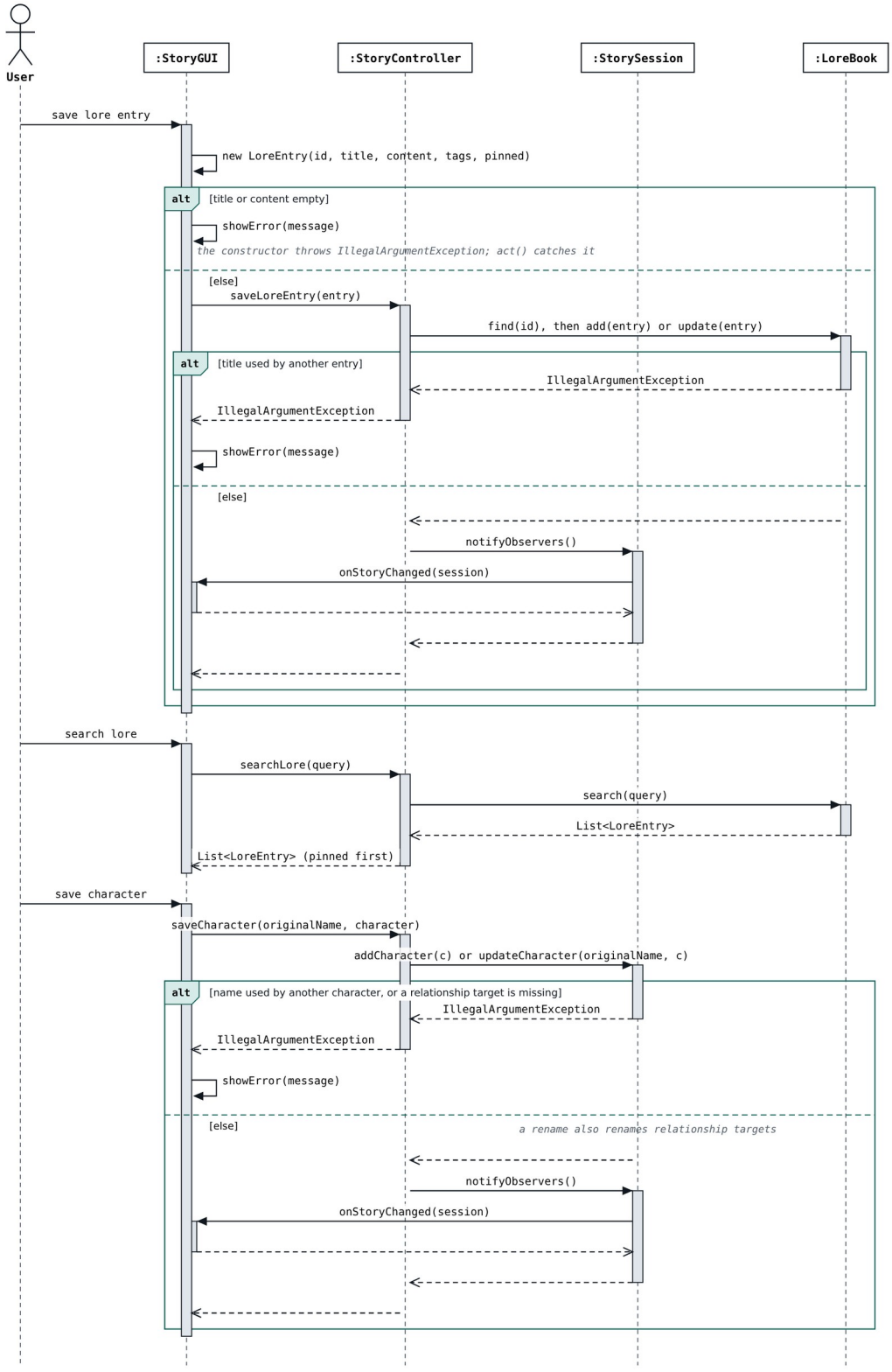


Figure 10. SD03 redrawn: invalid lore or characters throw an exception that the view catches and shows; the controller never calls showError.

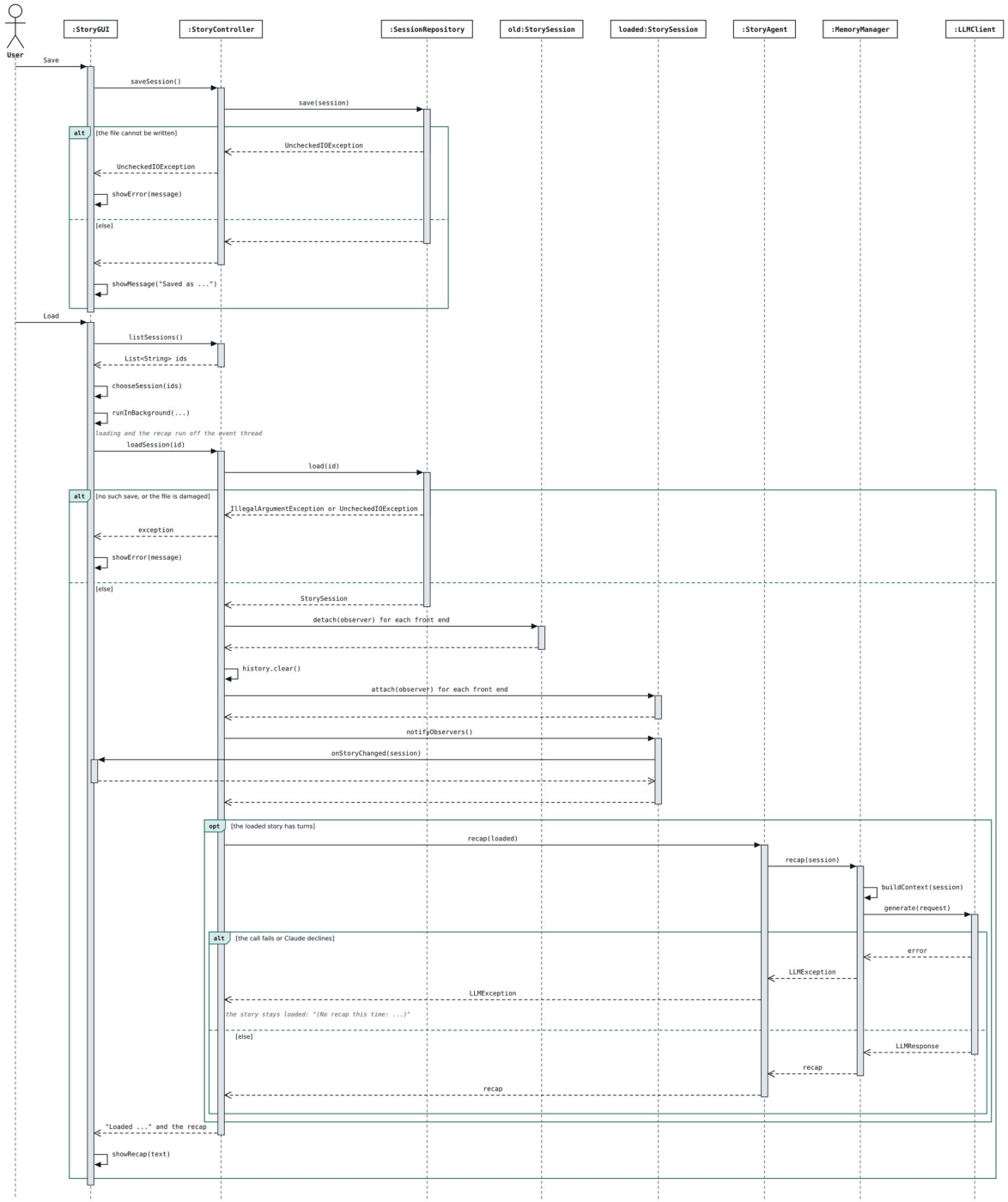


Figure 11. SD09 redrawn: save and load errors come back as exceptions; loading swaps the observers onto the loaded session, clears the undo history, and returns the "Loaded …" line followed by the model's recap (or the reason there is none).

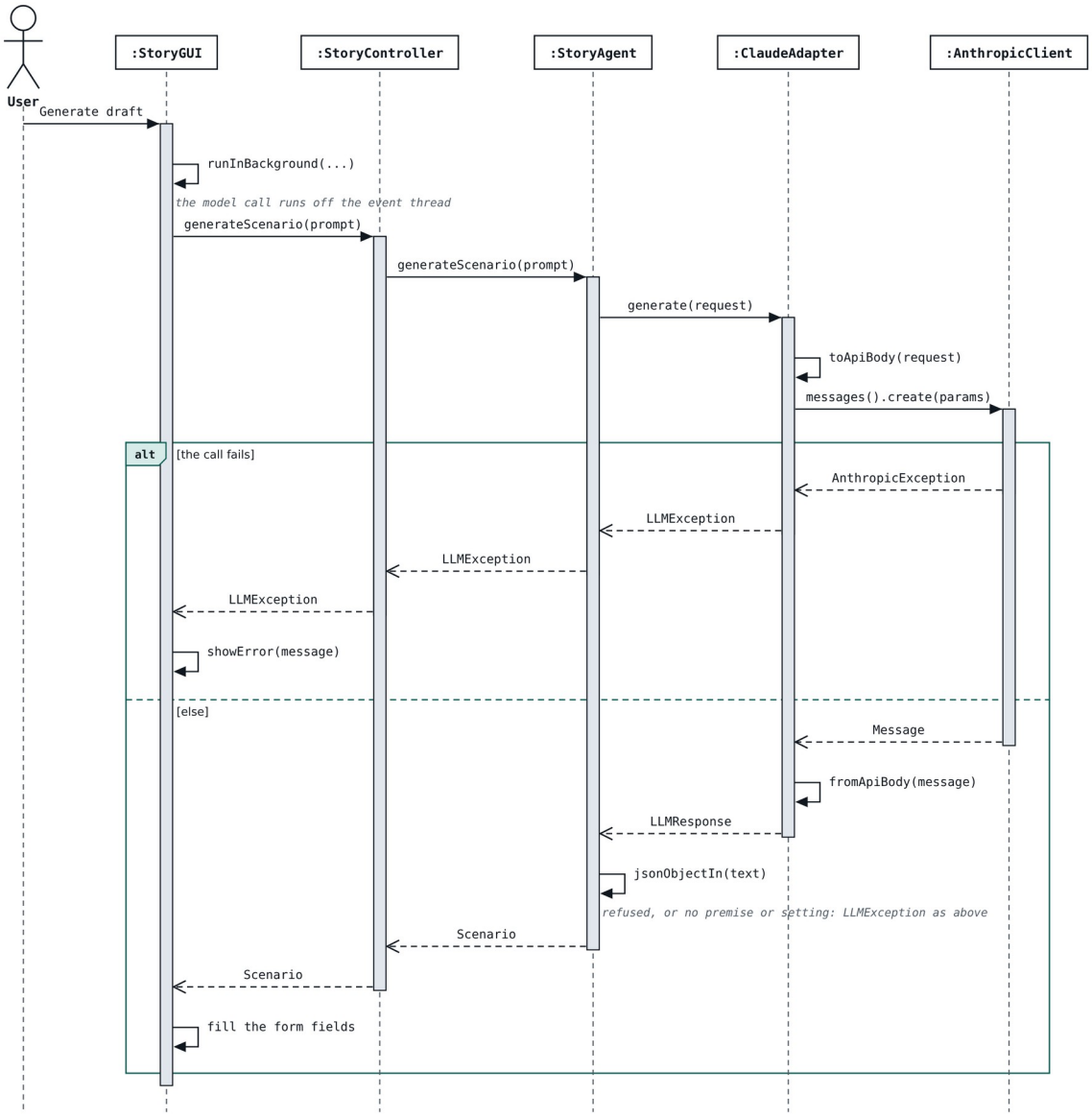


Figure 12. SD01 redrawn: the draft comes back as a Scenario or an exception that the view shows; the adapter calls the SDK's messages().create(params).

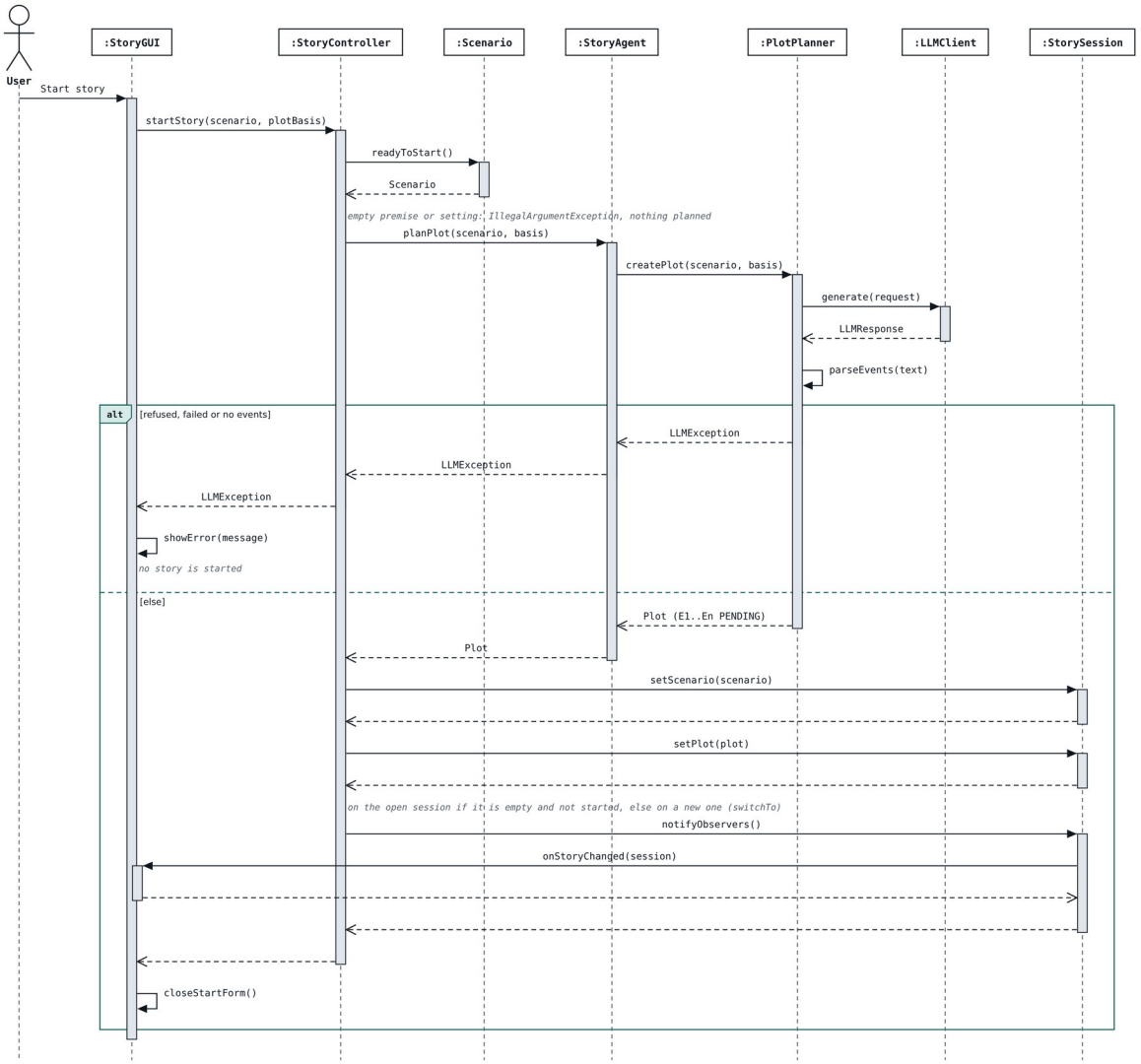


Figure 13. SD02 redrawn: startStory checks the scenario, plans the pivotal events first and keeps lore prepared in an empty session.

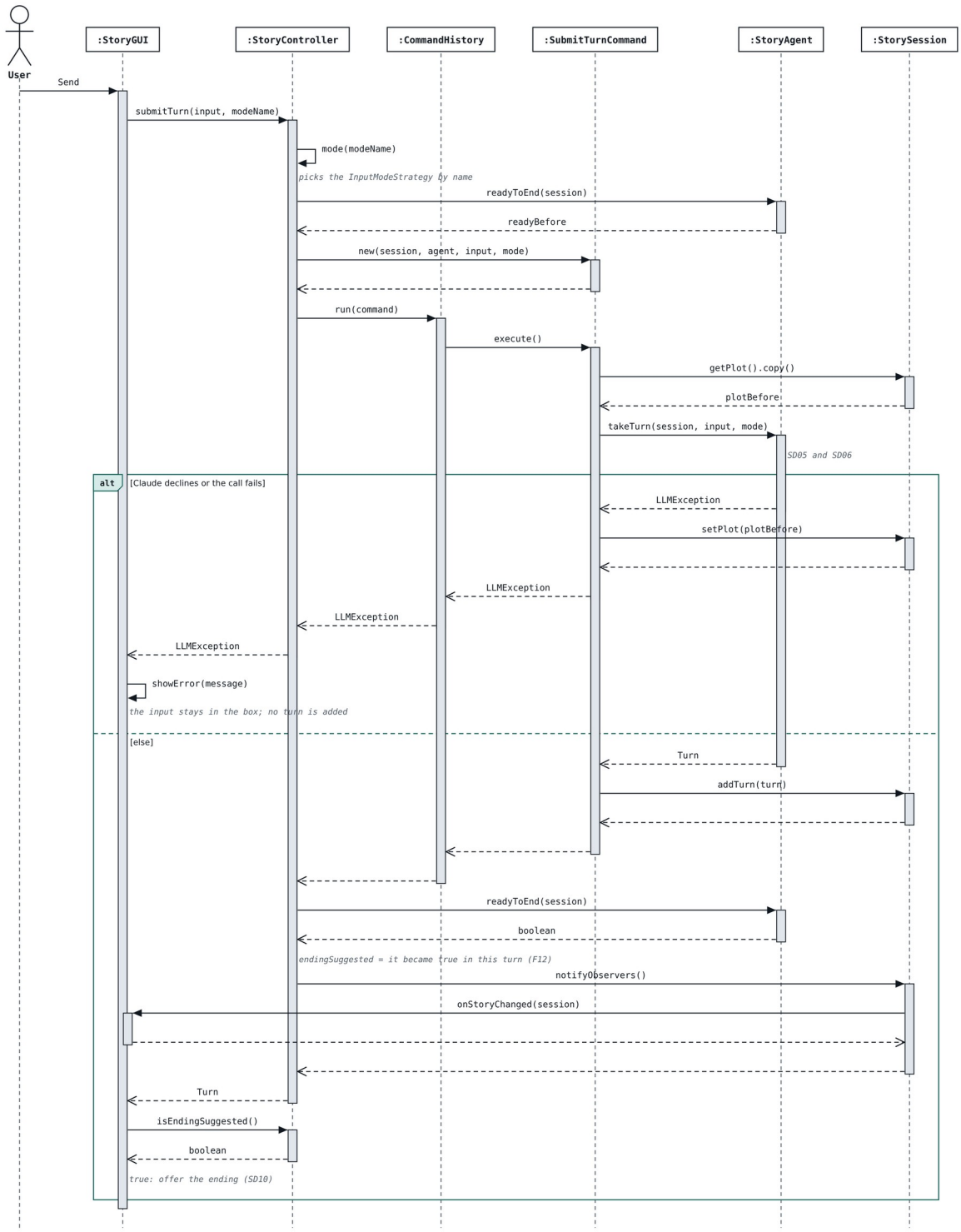


Figure 14. SD04 redrawn: the GUI runs the turn in the background through SubmitTurnCommand, redraws from onStoryChanged, and offers the ending when the plan completes.

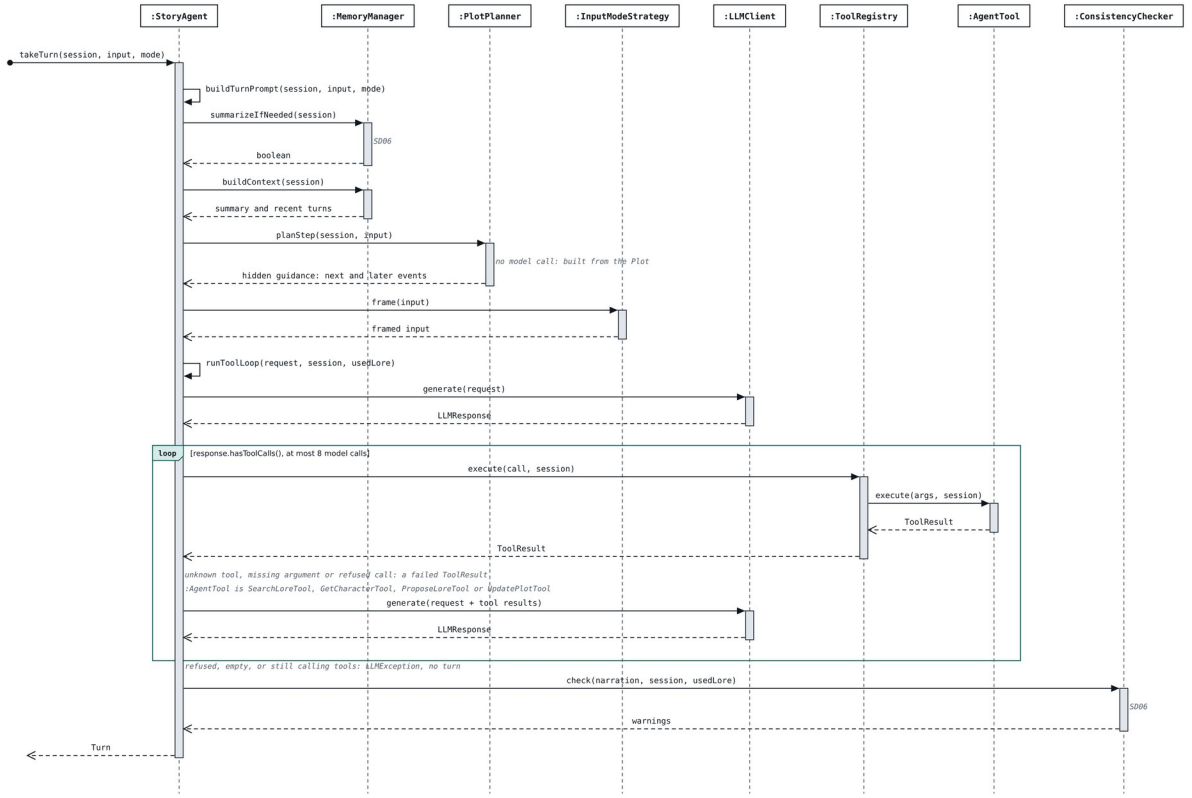


Figure 15. SD05 redrawn: summarize, build context and plan guidance (no model call), then the tool loop with update_plot, capped at 8 rounds, then the consistency check.

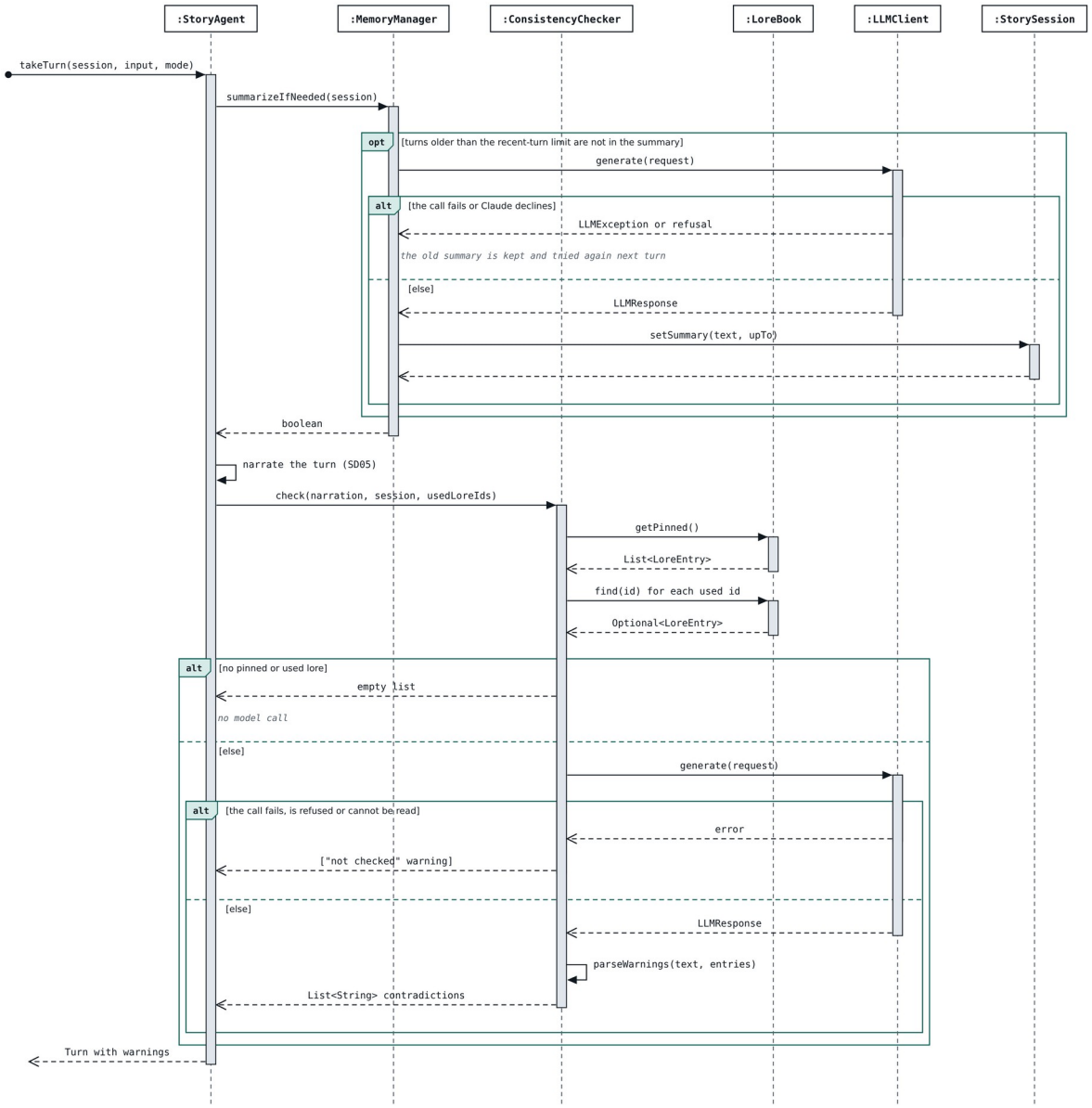


Figure 16. SD06 drawn from the code: a failed summary keeps the old one for the next turn; the check takes the lore the turn used and makes no model call when there is none.

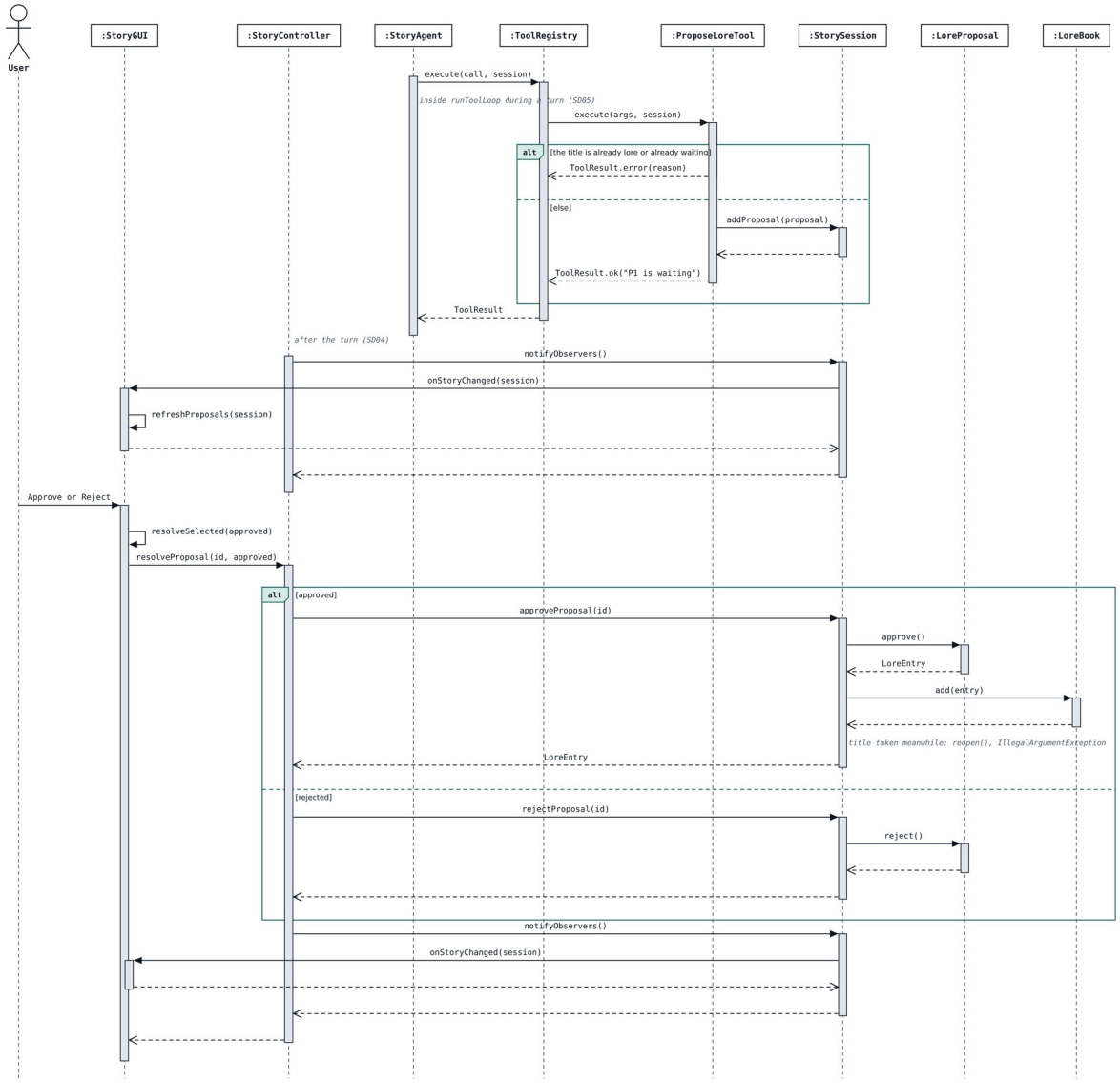


Figure 17. SD07 redrawn: no showProposal call; the proposals list is redrawn when observers are notified.

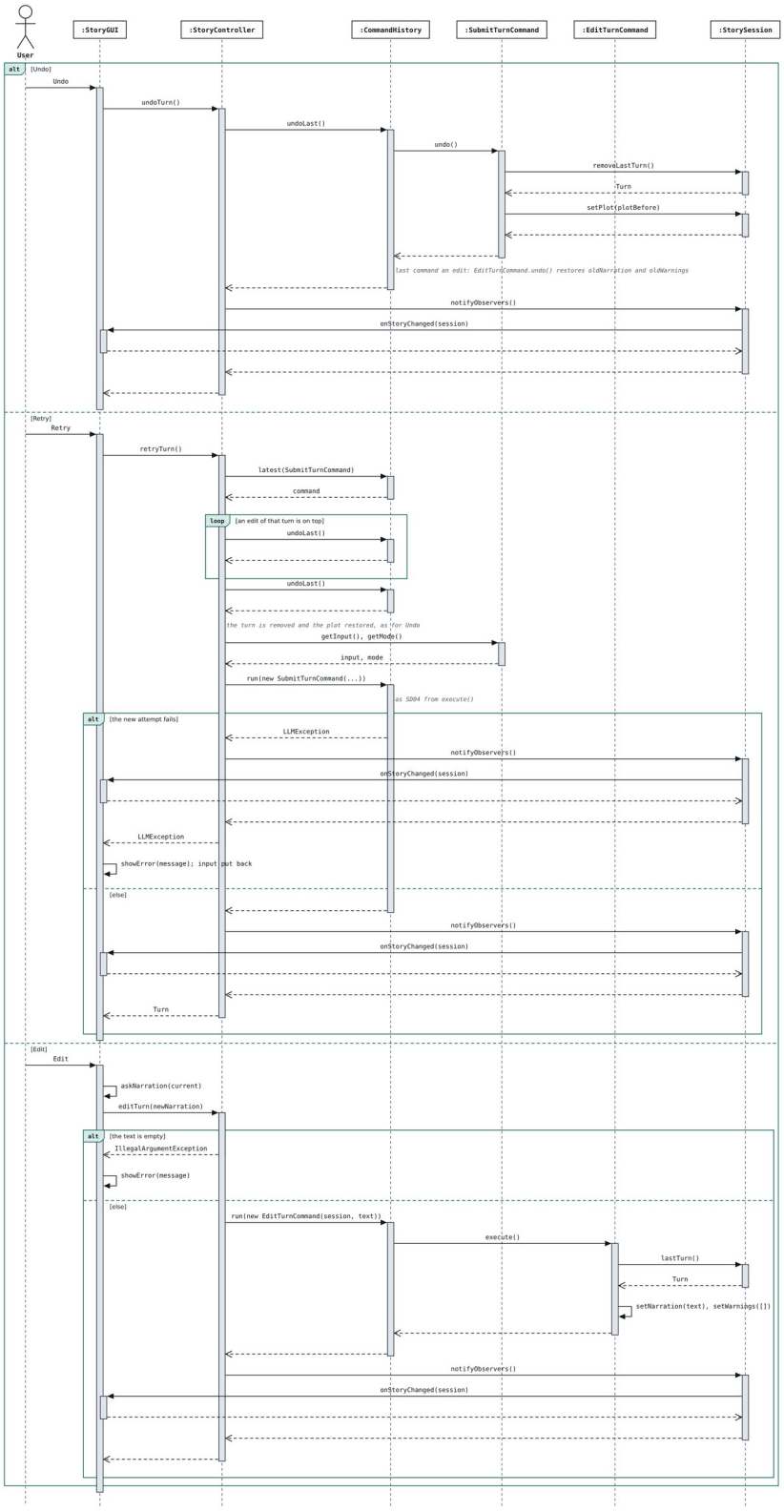


Figure 18. SD08 redrawn: retry first undoes edits made after the turn, then the turn, then runs it again with the same mode.

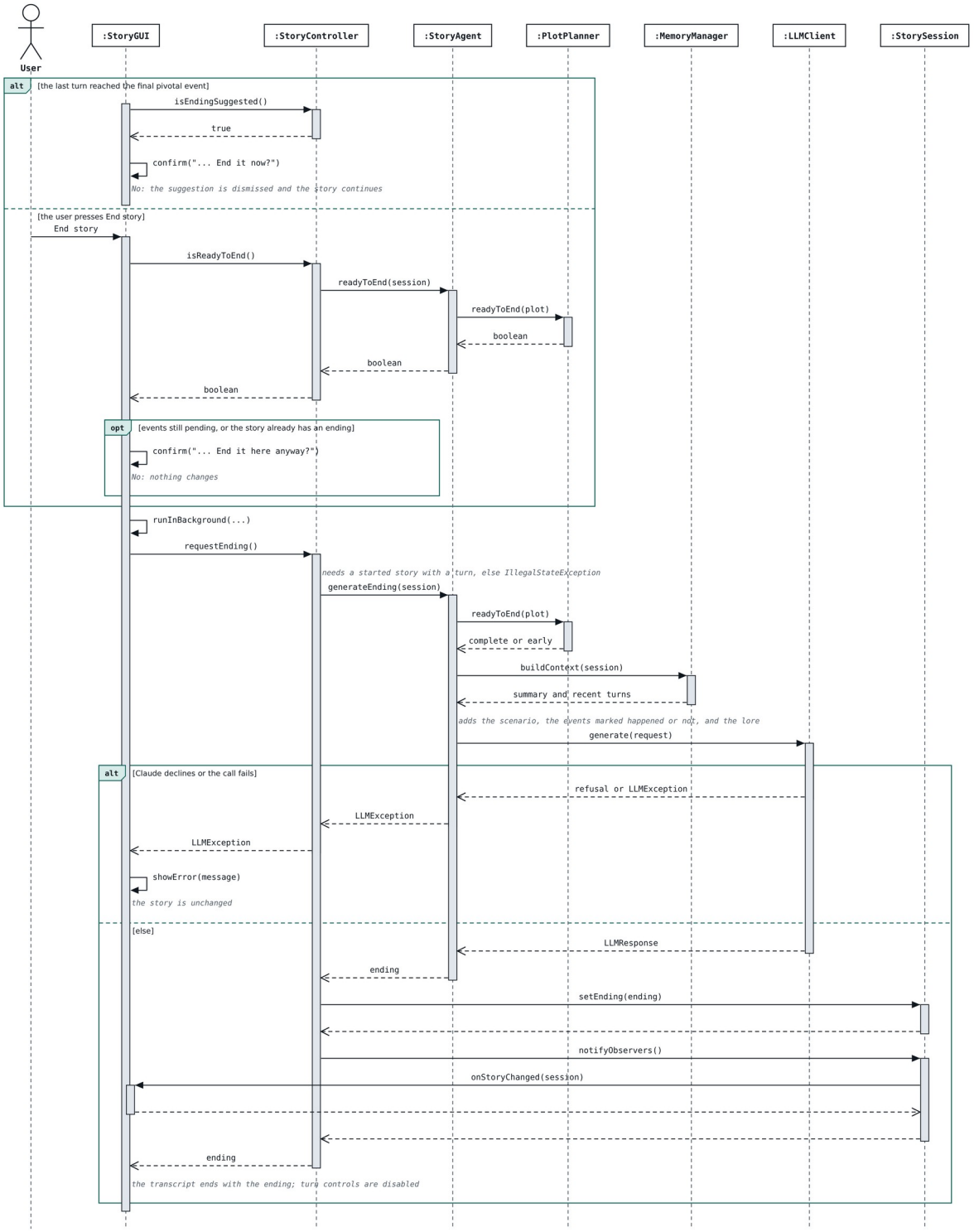


Figure 19. SD10 drawn from the code: confirmation for an early ending, generateEnding, the ending stored on the session and shown by both views.

4. Changes from Stage 1 and Rationale

Every feature, class or pattern that was added, replaced, removed or substantially modified relative to the Stage 1 design.

4.1 Summary of Changes

Original	Change	Reason	Impact
F03	Relationship stores the target name; StorySession gains update, remove and addRelationship	Save files without cycles; edit and remove were implied by UC03	UC03; Relationship, Character, StorySession
Model	Optional returns, detach, rejectProposal, setSummary; dropped events kept	Explicit empty cases and the counterparts F07 and F09 need	UC04 to UC06; StorySession, Plot, LoreProposal; Observer
F11	Missing, damaged and unwritable saves reported as exceptions; atomic writes	The design did not say how errors travel	UC07; SessionRepository, JsonSessionRepository
Adapter	LLMRequest/ LLMResponse carry LLMMessage, LLMToolCall, LLMToolResult, LLMToolSpec; SDK types instead of JSON strings	A tool-use loop needs roles, tool calls and results; thinking blocks must round-trip	UC01, UC04 to UC08; llm package; Adapter
Controller	Exceptions shown by the view; saveCharacter(originalName, c); edit/remove methods	Two views can be open; UC02/UC03 needed edit and remove	UC02, UC03, UC07; StoryController, StoryCLI, StoryGUI; MVC
F04	AgentTool gains getDescription; ToolCall gains id; ToolResult gains loreIds; schemas() returns LLMToolSpec	The API needs descriptions and call ids; the turn must record the lore it used	UC04; agent.tools; Facade
F04/F06	PlotPlanner.planStep builds the guidance from the plot; no second model call	Halves the cost and latency of each turn; the narrating model still decides	UC04; PlotPlanner, StoryAgent; SD05

Original	Change	Reason	Impact
F04	StoryAgent(llm, tools, planner, memory) and create(llm); runToolLoop capped at 8 calls and records lore used	A loop needs a bound; F04 records the lore used	UC04; StoryAgent; Facade, Strategy
F01/F05	startStory plans first, keeps lore prepared in an empty session; Scenario.readyToStart; controller takes the agent	Lore and characters can be prepared before a scenario; a failed plan must change nothing	UC01; StoryController, Scenario, StoryAgent; SD02
F04/F10	SubmitTurnCommand restores the plot on failure, has getMode/getResult; retry undoes edits first; CommandHistory.latest/ peekLast	After an edit the last command is not the turn; retry reuses the strategy	UC04, UC06; controller; Command; SD08
F10	Edit works whenever there is a turn, including after a load	Replacing text needs nothing lost at load time	UC06; EditTurnCommand, StoryController; Command
F07	ProposeLoreTool refuses titles already in the lore book or pending; listPendingProposals; lists redrawn on notify	Avoids re-proposing known facts; same refresh path as every other list	UC05; ProposeLoreTool, StoryController; SD07
F06	One update_plot tool with an action (reached / revise); revisions hold 1 to 6 events and stop once the plan is complete; planStep lists every pending event	The API gives one tool per name; limits stop an endless story; a revision must know what it replaces	UC04; UpdatePlotTool, PlotPlanner; SD05
F08	check(narration, session, usedLoreIds); at most 5 warnings; a failed check is one warning; an edit clears the warnings	The session does not know the used lore until the turn is added; warnings describe the agent's text	UC04, UC06; ConsistencyChecker, StoryAgent, Turn, EditTurnCommand; SD06

Original	Change	Reason	Impact
F09	summarizeIfNeeded runs at the start of a turn; MemoryManager takes the LLMClient; uncovered turns are sent after a failure	A summary at the end of a turn would leave one turn out of both the summary and the window	UC04; MemoryManager, StoryAgent; SD05, SD06
F09/F11	The recap follows a "Loaded …" line; a failed recap is explained; GUI shows it in a panel	The user sees which story opened and why there is no recap	UC07; StoryController, StoryAgent, MemoryManager, StoryGUI; SD09
F12	StorySession stores the ending; StoryAgent.readyToEnd; the plan is sent with each event marked	The closing passage must survive save and load; an early ending must not narrate events that never happened	UC08; StorySession, StoryAgent, PlotPlanner; SD10
F12	Ending suggested once, on the turn that completes the plan; an ended story takes no turns; End story rewrites the ending	A state-based prompt would repeat every turn; the ending stays the last thing in the story	UC08; StoryController, StoryGUI, StoryCLI; MVC; SD10
New (F04, F10, F12 display)	New turns and endings are revealed gradually (TranscriptPacer in the GUI, paced printing in the CLI)	Text appearing all at once was hard to follow; streaming would show text before the turn is checked and committed	UC04, UC08; StoryGUI, StoryCLI, TranscriptPacer (new); MVC
F11	The app opens to a story picker or "No stories detected"; stories are listed by title and save time; the story tab is named after the story	The user asked for it; ids do not say which story is which; the tab duplicated the Story menu	UC01, UC07; StoryController, SavedStory, StorySession, SessionRepository, StoryGUI, StoryCLI; MVC
New (GUI look)	StoryTheme gives the GUI one deep blue-purple palette whose shades stack as layers	The user asked for it; one class for the colours keeps them out of the layout code and testable	No use case; StoryTheme (new), StoryGUI.main; MVC (view)
F02, F03, F04, F07, F09 (GUI layout)	Lore, characters, proposals and the lore used this turn sit in a collapsible sidebar on the right; the recap is a card above the transcript	The user asked for it; the lore and the cast stay visible while the story is written	UC02, UC03, UC04, UC05, UC07 (display only); StoryGUI, StoryTheme.Card; MVC, Observer

4.2 Change Details

One table per change, copied from docs/changes-from-stage1.md.

Original ID	F03: Relationships point at a character by name
Original (Stage 1)	Relationship has an association to its target Character; StorySession has addCharacter and findCharacter, and the controller checks each relationship's target before calling Character.addRelationship(r).
Implemented	Relationship stores targetName (a String) instead of a Character reference. StorySession gains updateCharacter(originalName, character), removeCharacter(name) and addRelationship(fromName, r); the target checks (target exists, not itself) live in StorySession so every caller gets them.
What Changed and Why	a name reference has no cycles, so a session saves to JSON (F11) without object-identity bookkeeping. Edit and remove were implied by "manage characters" (UC03) but had no methods in the class diagram. When a character is renamed, updateCharacter rewrites the relationships that point at the old name; when one is removed, relationships pointing at it are removed too.
How the User Need Is Still Met	the user still adds, edits and removes characters and one-way relationships, with the same validation (empty name, duplicate name, missing target, self-relationship).
Affected Use Cases / Classes / Methods / Patterns	UC03; Relationship, Character, StorySession; MVC model only.
Diagrams Updated	figures/class-diagram/05-model-characters-and-events.png (Relationship field, StorySession methods). (to be redrawn under figures/stage2/ before submission)
Commit(s)	a80fa3a (https://github.com/Polarralop/EECS3311Project/commit/a80fa3a): feat(model): manage characters and relationships in StorySession

Original ID	Model: small additions to the Stage 1 model classes
Original (Stage 1)	StorySession has addTurn, removeLastTurn(): Turn, lastTurn(): Turn, addProposal, approveProposal, attach, notifyObservers; Plot.nextPending(): PivotalEvent.

Implemented	lastTurn() and Plot.nextPending() return Optional (there may be no turn or no pending event). StorySession also has detach(observer), rejectProposal(id), getPendingProposals(), setSummary(text, upTo) and nextTurnNumber(). Lore and proposal ids are assigned by the owning collection (L1, P1, ...). Plot.replacePending marks the old pending events DROPPED rather than deleting them.
What Changed and Why	Optional makes the empty case explicit instead of returning null. The extra methods are the obvious counterparts the features need (a rejected proposal in F07, the summary in F09). Keeping dropped events lets the plan's history be saved and inspected in tests.
How the User Need Is Still Met	no behaviour visible to the user changes.
Affected Use Cases / Classes / Methods / Patterns	UC04, UC05, UC06; StorySession, Plot, LoreProposal; Observer (subject gains detach).
Diagrams Updated	figures/class-diagram/04-model-story-and-lore.png, 05-model-characters-and-events.png. (to be redrawn under figures/stage2/ before submission)
Commit(s)	957b423 (https://github.com/Polarralop/EECS3311Project/commit/957b423): feat(model): notify StoryObserver on session changes

Original ID	F11: Save errors are exceptions from the repository
Original (Stage 1)	SessionRepository.save/load/listSessions with no error type; "a missing or damaged file produces an error message".
Implemented	load throws IllegalArgumentException for a missing save and UncheckedIOException for an unreadable or damaged one (including a file whose stored id does not match its name); save throws UncheckedIOException and writes through a temporary file so a failed save never leaves a half-written story. Ids are limited to letters, digits, - and _ because they become file names.
What Changed and Why	the design did not say how errors travel; standard unchecked exceptions keep the interface signatures as designed while letting the controller turn them into a message.
How the User Need Is Still Met	the user still gets an error message and stays where they were.
Affected Use Cases / Classes / Methods / Patterns	UC07; SessionRepository, JsonSessionRepository.
Diagrams Updated	none (signatures unchanged). (to be redrawn under figures/stage2/ before submission)

Commit(s)	0913872 (https://github.com/Polarralop/EECS3311Project/commit/0913872): feat(persistence): save and load sessions as JSON

Original ID	Adapter: structured messages and tool calls in the llm package
Original (Stage 1)	LLMRequest(systemPrompt, messages: List<String>, toolSchemas: List<String>), LLMResponse(text, refused) with hasToolCalls(); ClaudeAdapter.toApiBody(request): String, fromApiBody(json: String) and an external AnthropicApi.createMessage(jsonBody).
Implemented	LLMRequest holds List<LLMMessage> and List<LLMToolSpec> plus maxTokens; LLMResponse adds toolCalls, stopReason, token usage and the assistant message to append. New value types LLMMessage, LLMToolCall, LLMToolResult, LLMToolSpec, and LLMException for failures. toApiBody builds the SDK's MessageCreateParams and fromApiBody reads the SDK's Message; the adaptee is the anthropic-java SDK (client.messages().create). FakeLLMClient can also script failures and records requests.
What Changed and Why	a tool-use loop (F04) needs messages with roles, tool calls and tool results, which a list of strings cannot carry. The typed SDK replaces hand-written JSON, so the adapter converts between two object models rather than to and from strings. Each assistant reply keeps the SDK's own copy (opaque to everything but the adapter) so Claude's thinking blocks are sent back unchanged during a tool loop, as the API requires. These types live in llm so that package does not depend on agent.tools; the agent maps LLMToolCall to ToolCall.
How the User Need Is Still Met	the agent still talks to one LLMClient interface, and only ClaudeAdapter knows about Anthropic.
Affected Use Cases / Classes / Methods / Patterns	UC01, UC04 to UC08; LLMRequest, LLMResponse, ClaudeAdapter, FakeLLMClient, new LLMMessage, LLMToolCall, LLMToolResult, LLMToolSpec, LLMException; Adapter.
Diagrams Updated	figures/class-diagram/08-llm.png. (to be redrawn under figures/stage2/ before submission)
Commit(s)	cee9994 (https://github.com/Polarralop/EECS3311Project/commit/cee9994): feat(llm): send requests through ClaudeAdapter with tool-use support

Original ID	Controller: errors travel as exceptions, and extra management methods
Original (Stage 1)	on invalid input the controller "calls showError(message)" on the front end; saveCharacter(character) covers add and edit; the controller lists only the methods named in the sequence diagrams.

Implemented	controller methods throw IllegalArgumentException / IllegalStateException (or UncheckedIOException for save files) with a readable message and change nothing; the front end catches it and shows it (StoryCLI.printError, StoryGUI.showError). Validation rules live in the model classes, so every caller gets them. saveCharacter(originalName, character) takes the name being edited (null for a new character) so a rename can be told apart from a new character. Added: addObserver, newSession, getLoreEntry, removeLoreEntry, getCharacter, listCharacters, removeCharacter, addRelationship, removeRelationship, listSessions, canUndo. Until F09 lands, loadSession returns a short deterministic recap (counts of lore, characters and turns) instead of StoryAgent.recap(session).
What Changed and Why	the controller does not hold a reference to a particular front end (two can be open), so returning the error to the caller is simpler than calling back into one view. Edit and remove were part of UC02 and UC03 but had no controller methods.
How the User Need Is Still Met	the user sees the same error messages and nothing changes on an error.
Affected Use Cases / Classes / Methods / Patterns	UC02, UC03, UC07; StoryController, StoryCLI; MVC.
Diagrams Updated	figures/class-diagram/03-controller.png; SD03 and SD09 show showError called by the controller. (to be redrawn under figures/stage2/ before submission)
Commit(s)	c749429 (https://github.com/Polarralop/EECS3311Project/commit/c749429): feat(controller): expose lore, characters and save/load in StoryCLI

Original ID	F04: Tool types carry what the agent loop needs
Original (Stage 1)	AgentTool has getName(), getSchema() and execute(args, session); ToolCall(toolName, arguments); ToolResult(success, content); ToolRegistry.schemas(): List<String>.
Implemented	AgentTool also has getDescription(). ToolCall is a record with the model's call id as well. ToolResult is a record with loreIds, the lore entries the result showed the model. ToolRegistry.schemas() returns List<LLMToolSpec>. The registry reads each tool's required arguments from its schema and rejects a call missing one before the tool runs; an unknown tool or a tool that throws becomes a failed ToolResult.
What Changed and Why	The Messages API needs a description and an id per tool call, and F04's "record of which lore entries were used" needs the tool result to say which entries it returned.

How the User Need Is Still Met	The agent still reaches the story only through validated tool calls, and an invalid request is reported back to the agent while the turn continues (UC04 4b).
Affected Use Cases / Classes / Methods / Patterns	UC04; AgentTool, ToolCall, ToolResult, ToolRegistry, SearchLoreTool, GetCharacterTool; Facade (subsystem).
Diagrams Updated	figures/class-diagram/07-agent-tools.png (redrawn under figures/stage2/).
Commit(s)	feat(tools): register SearchLoreTool and GetCharacterTool in ToolRegistry

Original ID	F04/F06: Steering guidance is built without a separate model call
Original (Stage 1)	During a turn, PlotPlanner.planStep(session, input) sends the input, the recent story and the next pending event to the model and returns guidance, which then goes into the narration request.
Implemented	planStep builds the guidance from the plot itself: how many events are reached, the next pending event and an instruction to nudge back gradually; empty when there is no plan. The narration call receives it in a section marked hidden and decides whether to continue, nudge or (with UpdatePlotTool, F06) revise.
What Changed and Why	A second model call per turn would double the cost and latency of every turn for a decision the narrating model can make while it already has the guidance, the input and the story. The decision is still the model's, so F06 stays an AI feature.
How the User Need Is Still Met	The story is still steered towards the hidden events, and the events are never shown to the user.
Affected Use Cases / Classes / Methods / Patterns	UC04 step 6; PlotPlanner.planStep, StoryAgent.takeTurn; Facade.
Diagrams Updated	SD05 (no model call inside planStep).
Commit(s)	feat(agent): run the tool-use loop in StoryAgent.takeTurn

Original ID	F04: StoryAgent wiring and the turn loop limit
Original (Stage 1)	StoryAgent composes PlotPlanner, ConsistencyChecker and MemoryManager and uses ToolRegistry and LLMClient; runToolLoop(request, session): LLMResponse repeats while the response has tool calls.

Implemented	StoryAgent(llm, tools, planner, memory) plus StoryAgent.create(llm), which registers the standard tools. runToolLoop(request, session, usedLore) collects the lore ids of successful lookups and stops after 8 model calls with an error. The system prompt is read from src/main/resources/prompts/system.txt. ConsistencyChecker joins the constructor with F08.
What Changed and Why	The loop needs a bound; recording the lore used is the F04 output "a record on the turn of which lore entries were used".
How the User Need Is Still Met	As designed; a turn that cannot finish shows a message and adds nothing (UC04 7a).
Affected Use Cases / Classes / Methods / Patterns	UC04; StoryAgent; Facade, Strategy.
Diagrams Updated	figures/class-diagram/06-agent.png (redrawn under figures/stage2/).
Commit(s)	feat(agent): run the tool-use loop in StoryAgent.takeTurn

Original ID	F01/F05: Starting a story keeps lore prepared beforehand
Original (Stage 1)	StoryController.startStory(scenario, plotBasis) checks the mandatory fields, has the events planned and creates the StorySession that holds the scenario.
Implemented	startStory calls Scenario.readyToStart() (premise and setting mandatory; an empty title becomes the first words of the premise), then StoryAgent.planPlot, and only then changes anything. An empty, unstarted open session receives the scenario and plot; otherwise a new session is created. StoryController takes the StoryAgent in its constructor; isStarted() tells the front ends whether turns are possible. StoryCLI.createController() reads .env; without an API key the app opens and model features report the missing key.
What Changed and Why	F02 and F03 already let the user prepare lore and characters before any scenario exists, so starting must not throw that away. Planning before any change means a failed plan leaves everything as it was.
How the User Need Is Still Met	The user still writes the scenario, optionally a plot basis, and starts; the plan stays hidden.
Affected Use Cases / Classes / Methods / Patterns	UC01; StoryController, Scenario, StoryAgent, PlotPlanner, StoryCLI, StoryGUI; Facade, MVC.
Diagrams Updated	03-controller.png, 04-model-story-and-lore.png; SD02.
Commit(s)	feat(agent): plan hidden pivotal events before the story starts

Original ID	F04/F10: Turn commands and retry details
Original (Stage 1)	SubmitTurnCommand(input, mode, plotBefore, result) with execute, undo, getInput; CommandHistory.run/undoLast/canUndo; StoryController.retryTurn() calls undoLast(), reads the input and runs a new command.
Implemented	SubmitTurnCommand(session, agent, input, mode) also has getMode() and getResult(), and puts the plot copy back if the agent fails (lore proposals it raised stay pending, as for an undone turn). CommandHistory gains latest(type) and peekLast(). retryTurn() finds the latest SubmitTurnCommand, undoes any edits of that turn, undoes the turn and runs a new command with the same input and mode; observers are notified even when the retry fails. StoryController gains modeNames() and isStarted(); turns need a started story.
What Changed and Why	Stage 1 assumed the last command is always the turn, but after an edit the last command is the EditTurnCommand; regenerating the turn must discard the edit too. getMode() is how retry reuses the same strategy.
How the User Need Is Still Met	Undo removes the turn and restores the events; retry regenerates it from the same input; a failed retry leaves the turn undone and the input is shown again.
Affected Use Cases / Classes / Methods / Patterns	UC04, UC06; SubmitTurnCommand, CommandHistory, StoryController, StoryCLI; Command, Strategy.
Diagrams Updated	03-controller.png; SD08.
Commit(s)	feat(controller): submit turns through SubmitTurnCommand and CommandHistory

Original ID	F10: A loaded turn can be edited
Original (Stage 1)	The Undo, Retry and Edit buttons are available only while CommandHistory.canUndo() is true; EditTurnCommand(newNarration, oldNarration).
Implemented	StoryController.editTurn(newNarration) works whenever the story has a turn, including right after a load; the edit is then in the history and can be undone. Undo and retry still need a turn taken since the story was opened. EditTurnCommand(session, newNarration) keeps the edited Turn and its oldNarration.
What Changed and Why	Replacing text needs nothing that was lost at load time (no plot copy, no original input), so there is no reason to forbid it.
How the User Need Is Still Met	Edit, undo of the edit, and the empty-text and cancel cases behave as in UC06.

Affected Use Cases / Classes / Methods / Patterns	UC06; EditTurnCommand, StoryController, StoryCLI; Command.
Diagrams Updated	03-controller.png.
Commit(s)	feat(controller): edit a past turn with EditTurnCommand

Original ID	F07: Proposal checks and the pending list
Original (Stage 1)	ProposeLoreTool.execute(args, session) checks that the title and content are present and calls StorySession.addProposal; StoryController.resolveProposal(id, approved); the front end calls showProposal(proposal).
Implemented	As designed, plus: the tool refuses a title that the lore book or another pending proposal already has, and StoryController.listPendingProposals() gives the front ends the list. A proposal raised before a failed model call still triggers a notification so it appears. The front ends redraw the list in onStoryChanged instead of a separate showProposal call.
What Changed and Why	Without the title check the model tends to re-propose facts it has just looked up, and approval would fail on the lore book's duplicate-title rule anyway. Redrawing from the session is how every other list already works (Observer).
How the User Need Is Still Met	The agent proposes, the user approves or rejects each proposal, and only approved proposals become lore.
Affected Use Cases / Classes / Methods / Patterns	UC05; ProposeLoreTool, StoryController, StoryCLI; Observer, MVC.
Diagrams Updated	SD07.
Commit(s)	feat(tools): raise lore proposals and resolve them in StoryController

Original ID	F06: UpdatePlotTool takes an action, and the guidance lists every pending event
Original (Stage 1)	UpdatePlotTool.execute(args, session) calls Plot.markReached(eventId) or Plot.replacePending(events); PlotPlanner.planStep(session, input) returns guidance about the next pending event; a call naming an unknown event is a failed ToolResult.

Implemented	one update_plot tool with an action argument: "reached" with event_id, or "revise" with new_events (a JSON array; plain text with one event per line is accepted too). New events get the next free ids (E4, E5, ... after E3, dropped events included), a revision holds 1 to 6 events (UpdatePlotTool.MAX_NEW_EVENTS), and revising is refused once every event is reached, so the story moves to an ending (F12) instead of growing forever. An event reached out of order is accepted and the result says which earlier one is still pending. planStep now also lists the later pending events and tells the model when to call update_plot, because a revision has to know what it is replacing.
What Changed and Why	the Messages API gives the model one tool per name, and an action enum keeps both plot operations behind the single UpdatePlotTool class the design has. The limits keep a model from rewriting the plan into an endless story.
How the User Need Is Still Met	when the user goes off-script the narration nudges back or the plan is revised to fit; the events stay hidden in both front ends, and undo or a failed turn puts the plan back (F10).
Affected Use Cases / Classes / Methods / Patterns	UC04 step 6; UpdatePlotTool, PlotPlanner.planStep, StoryAgent.create (registers the tool), prompts/system.txt; Facade (subsystem).
Diagrams Updated	Figure 7 (07-agent-tools.png); Figure 15 (SD05).
Commit(s)	feat(tools): steer the hidden plot with UpdatePlotTool

Original ID	F08: The check takes the lore the turn used, and an edit clears the warnings
Original (Stage 1)	ConsistencyChecker.check(narration, session): List<String> compares the narration with the pinned facts "together with the lore entries the turn used"; the agent stores the list in Turn.warnings; a failed check returns the turn "with a warning saying that it was not checked"; the front end shows warnings in showNarration(turn).
Implemented	check(narration, session, usedLoreIds): the ids of the entries looked up during the turn are passed in, because the session does not know them until the turn is added. The model answers with a JSON list of {entry, problem}; each becomes a warning such as Contradicts L1 "Orrin" (canon): ..., at most 5 per turn. A failed call, a refusal or an unreadable answer all give one warning starting "This turn was not checked against the lore". StoryAgent takes the checker in its constructor. Turn.setWarnings is added, and EditTurnCommand clears a turn's warnings when the user replaces its narration and restores them on undo. The front ends redraw the warnings from the turns in onStoryChanged rather than through a showNarration call.

What Changed and Why	the signature needs the used lore to do what the design describes. The warnings are about the agent's text; once the user has written their own, an old warning would point at words that are gone.
How the User Need Is Still Met	contradictions with pinned canon or the lore in play are flagged under the turn and never block it; the user can retry or edit (F10).
Affected Use Cases / Classes / Methods / Patterns	UC04 steps 8, 8a, 8b; UC06 1b; ConsistencyChecker, StoryAgent, Turn, EditTurnCommand, StoryCLI, StoryGUI, prompts/consistency-check.txt; Facade, Command.
Diagrams Updated	Figures 3, 4 and 6 (03-controller, 04-model-story-and-lore, 06-agent); Figure 16 (SD06).
Commit(s)	feat(agent): flag contradictions with pinned lore in ConsistencyChecker

Original ID	F09: Older turns are summarized at the start of a turn, not the end
Original (Stage 1)	at the end of takeTurn the agent calls MemoryManager.summarizeIfNeeded(session); buildContext(session) returns the summary and the most recent turns up to recentTurnLimit; MemoryManager has no constructor arguments in the class diagram.
Implemented	takeTurn calls summarizeIfNeeded first, then buildContext. MemoryManager(llm) / MemoryManager(llm, recentTurnLimit) take the model client. summarizeIfNeeded returns whether the summary changed. buildContext sends the summary followed by every turn the summary does not cover, normally exactly the last recentTurnLimit turns; when summarizing has failed it also sends the uncovered older turns, up to twice the limit.
What Changed and Why	the turn being narrated is added to the session by SubmitTurnCommand only after takeTurn returns, so a summary written at the end of takeTurn would stop one turn short of the next turn's window and that turn would be in neither the summary nor the recent turns. Summarizing at the start sees the session as the narration will, so the summary always ends where the recent turns begin. Sending uncovered turns on failure keeps the "try again after the next turn" fallback from losing a turn meanwhile.
How the User Need Is Still Met	long stories send a summary plus the recent turns; a failed summary is retried on the next turn and the story stays coherent in between.
Affected Use Cases / Classes / Methods / Patterns	UC04 steps 5 and 9a; MemoryManager, StoryAgent, prompts/summarizer.txt; Facade.
Diagrams Updated	Figure 6 (06-agent.png); Figures 15 and 16 (SD05, SD06).
Commit(s)	feat(agent): summarize older turns in MemoryManager

Original ID	F09/F11: The recap is shown with what was loaded, in a panel on the Story tab
Original (Stage 1)	StoryController.loadSession(id) calls StoryAgent.recap(session), which delegates to MemoryManager.recap(session), and returns the recap; the story screen shows it at the top; a failed recap means the story loads without one. Week 1 returned a deterministic line (counts of lore, characters and turns) until F09 was built.
Implemented	as designed, and loadSession returns the deterministic "Loaded ..." line followed by "The story so far: <recap>". A story without turns gets no recap and no model call. When the recap fails or is refused, the story is loaded and the message ends with "(No recap this time: <reason>)". The recap is written from the scenario, the summary and the recent turns (prompts/recap.txt), never the hidden plan. StoryGUI loads in the background and shows the message in a "Welcome back" panel at the top of the Story tab, which hides when another story is opened; StoryCLI prints it.
What Changed and Why	telling the user why there is no recap is more helpful than silently showing none, and the counts line confirms which story was opened. A panel instead of a dialog keeps the recap in view while the user reads the transcript.
How the User Need Is Still Met	a reopened story shows a recap at the top of the story screen and play continues from the last turn; a failed recap never stops the load.
Affected Use Cases / Classes / Methods / Patterns	UC07 1a, 1d; StoryController.loadSession, StoryAgent.recap, MemoryManager.recap, StoryGUI (recap panel, showRecap), StoryCLI (load); Facade, MVC.
Diagrams Updated	Figures 2 and 6 (02-ui, 06-agent); Figure 11 (SD09).
Commit(s)	feat(controller): show a model-written recap when a story is loaded

Original ID	F12: The ending is stored on the session
Original (Stage 1)	StoryAgent.generateEnding(session) returns the closing passage to the controller and the front end shows it at the end of the story; PlotPlanner.readyToEnd(plot) uses Plot.isComplete(). The model has no field for the ending.
Implemented	StorySession gains ending with getEnding(), setEnding(text) and isEnded(), so the ending is saved and loaded with the story (F11). StoryAgent.readyToEnd(session) is the agent's own entry point to PlotPlanner.readyToEnd, so the controller does not reach past the facade. generateEnding sends the plan with each event marked as happened or not (dropped events left out), says whether the plan was completed or the user ended early, and adds every lore entry and the story context.

What Changed and Why	UC08's postcondition is "the story has a closing passage", which has to survive a save and load like every other part of the story. Marking the events lets an early ending avoid narrating events that never happened.
How the User Need Is Still Met	the agent writes an ending from the pivotal events, the lore and the story so far, and the ending stays with the story.
Affected Use Cases / Classes / Methods / Patterns	UC08; StorySession, StoryAgent, PlotPlanner, prompts/ending.txt; Facade, MVC (model).
Diagrams Updated	Figures 4 and 6 (04-model-story-and-lore, 06-agent); Figure 19 (SD10).
Commit(s)	feat(agent): write the story's ending in StoryAgent.generateEnding

Original ID	F12: How the ending is suggested, and what an ending closes
Original (Stage 1)	an End Story button calls StoryController.requestEnding(); "when the agent judges that the story is ready to end, a prompt suggests it" (once PlotPlanner.readyToEnd(plot) is true); an early request asks the user to confirm; on failure the controller calls showError(message) and the story is unchanged. Nothing is said about turns after an ending.
Implemented	requestEnding() returns the ending and stores it with StorySession.setEnding. The controller adds four queries for the front ends: canEnd() (a started story with a turn), isReadyToEnd() (no pending events; when false the front end asks to confirm), isEndingSuggested() (true only right after the turn or retry that reached the last pivotal event, so a dismissed suggestion is not repeated every turn) and isEnded(). An ended story takes no more turns, undo, retry or edit; End story on an ended story asks to confirm and writes a new ending in its place. Errors reach the front end as exceptions, as for every other controller method. The GUI button reads "Rewrite ending" once the story has ended, and the CLI command is end.
What Changed and Why	the design leaves "ready to end" as a state, which would prompt after every turn once the plan is complete; suggesting on the transition gives the user one prompt to accept or dismiss, as UC08 1a describes. Blocking turns keeps the closing passage the last thing in the story, and rewriting gives the user a way to try again without undo history (which is not saved).
How the User Need Is Still Met	the user can end the story at any time (confirming when the plot is unfinished), the agent offers to end once the plan is complete, and the ending is shown at the end of the story in both front ends and kept when the story is saved.
Affected Use Cases / Classes / Methods / Patterns	UC08 (main flow, 1a, 1b, 2a); StoryController, StoryCLI (end, offerEnding), StoryGUI (endStory, confirm, offerEnding); MVC, Observer.

Diagrams Updated	Figures 2 and 3 (02-ui, 03-controller); Figure 19 (SD10).
Commit(s)	feat(ui): end the story from StoryGUI and StoryCLI

Original ID	New: New story text is revealed gradually in both front ends
Original (Stage 1)	after a turn the controller notifies the observers and the front end shows the new narration (SD04); how fast it appears is not specified, so in week 3 the whole turn appeared at once.
Implemented	a new package-private class ui/TranscriptPacer shows the GUI transcript. When the transcript only grows in the same story (a new turn, a retry, the ending), show(text, pace) reveals the new part a few characters every 20 ms (tick()) at StoryGUI.CHARS_PER_SECOND (300 characters a second), sped up so a reveal takes at most 10 seconds; clicking the transcript or pressing Send calls skip(). Loading, undoing and editing replace the text at once. StoryCLI takes the same speed in a new constructor StoryCLI(controller, in, out, charsPerSecond) and prints a new turn or ending gradually in printText; turns and the old constructor (used by tests) print at once.
What Changed and Why	the user asked for the story to appear at a readable pace rather than all at once. Real streaming from the Messages API was considered and not used: a turn is a tool-use loop whose narration is checked by ConsistencyChecker and committed by SubmitTurnCommand before it is shown, so streaming would show text that a failed or retried turn later takes back, and it would change LLMClient, StoryAgent and SD04. Pacing the display keeps every turn validated first and leaves the model, agent and controller unchanged.
How the User Need Is Still Met	every turn is still complete, checked and undoable before it is shown; only the display is paced.
Affected Use Cases / Classes / Methods / Patterns	UC04 (story turn) and UC08 (ending), the display step; StoryGUI (refreshStory, submitInput, CHARS_PER_SECOND), TranscriptPacer (new), StoryCLI (constructor, printTurn, printEnding, printText); MVC (view only), Observer (the view still refreshes from onStoryChanged).
Diagrams Updated	Figure 2 (02-ui).
Commit(s)	feat(ui): reveal new story text gradually in both front ends

Original ID	F11: The app opens to a story picker, and stories are offered by title
Original (Stage 1)	UC07 has the user choose a saved session from a list (StoryController.listSessions() returns the ids) and loadSession(id) opens it. What the app shows on launch is not specified; in week 3 the GUI opened to empty tabs and the CLI to a help line.

Implemented	StoryController.listSavedStories() returns the saved stories, most recently saved first, as SavedStory(id, title, savedAt) records (new, controller package), loading each save through SessionRepository.load (a damaged save is listed as "(cannot be read)" and loading it still reports why) and asking the new SessionRepository.lastSaved(id) when it was saved (JsonSessionRepository reads the save file's modification time). The GUI shows each story as its title followed by "Saved Nov 9, 2026, 4:27 AM" in grey (StoryGUI.SavedStoryRenderer, savedLabel), without the id; the Load dialog shows "title (saved …)"; the CLI keeps the id, which load <id> needs, and adds the save time. StorySession.getTitle() gives the scenario's title, or StorySession.UNTITLED ("Untitled story") for a story prepared before its scenario. StoryCLI opens to "Select a story:" with each id and title and how to load one, or "No stories detected. Type \"start\" to create a story."; sessions also prints titles. StoryGUI now opens to a start screen instead of empty tabs (a CardLayout with a "home" card and the tabs): "Select story" with the saved stories by title, the newest selected, Open (or a double-click) and Create story…, or "No stories detected" with only Create story…, which opens the Start story form. The tabs appear once a story is open or started. The first tab, which was also labelled "Story" like the menu next to it, is now named after the open story (StorySession.getTitle(), cut to 40 characters with the whole title in its tooltip), and the line above the transcript only shows while the story has no scenario. Story > Load… lists titles too.
What Changed and Why	the user asked for the app to open to a story picker, or to "No stories detected" with a way to create one, and for the duplicate "Story" tab to show the story's title instead; ids such as story-20261108-101500 do not say which story is which.
How the User Need Is Still Met	loading still goes through loadSession(id) with its recap (F09) and fresh undo history; only how the user finds the story changed.
Affected Use Cases / Classes / Methods / Patterns	UC01 (start from the start screen), UC07 (choose a session); StoryController.listSavedStories, StorySession.getTitle, StoryCLI (printStartScreen, listSessions), StoryGUI (refreshHome, openSelectedStory, load, chooseSession, refreshStory, SavedStoryRenderer, savedLabel), SavedStory (new), SessionRepository.lastSaved, JsonSessionRepository.lastSaved; MVC.
Diagrams Updated	Figures 2, 3, 4 and 9 (02-ui, 03-controller, 04-model-story-and-lore, 09-persistence). SD09 still shows the user choosing from the list; only what the list shows changed.
Commit(s)	feat(controller): list saved stories by title and open the CLI to them; feat(ui): open StoryGUI to a story picker and name the tab after the story; feat(ui): show when each saved story was saved instead of its id

Original ID	New: The GUI has its own colour palette (StoryTheme)

Original (Stage 1)	the report does not specify how the GUI looks; StoryGUI used Swing's default Metal look (grey panels, blue highlights).
Implemented	a new package-private class ui/StoryTheme holds the palette, one deep blue-purple and its shades: BASE (the window), INSET (text boxes and lists), SURFACE (the story column and dialogs), SIDEBAR, RAISED (cards and buttons), LINE (borders), SELECTION, ACCENT (the main button of a screen) and ACCENT_SOFT, with TEXT, MUTED and DISABLED for text. The shades get lighter from the back layer to the front one, so a panel that sits on another reads as a layer above it. StoryTheme.install() applies the palette to every Swing component through a Metal theme (MetalLookAndFeel.setCurrentTheme) plus a few UIManager defaults (flat borders, padding, plain fonts); StoryGUI.main calls it before building the window. Helpers style the components that stand out: primary(button) / plain(button) (the accent only while the button can be pressed), quiet(button) (a text link), sectionLabel(text), layer(component, colour) and the rounded StoryTheme.Card panel.
What Changed and Why	the user asked for a deep blue-purple main colour with shades for layering. Keeping the colours in one class means no colour is typed into StoryGUI's layout code and the palette can be checked by tests (StoryThemeTest: the layers are one hue from dark to light, text contrast is at least 7:1 on every layer).
How the User Need Is Still Met	display only. StoryTheme never reads or changes the story; every action still goes through StoryController. The CLI is unchanged.
Affected Use Cases / Classes / Methods / Patterns	none of the use cases; StoryTheme (new), StoryGUI.main; MVC (view only).
Diagrams Updated	figures/class-diagram/02-ui.png (StoryTheme); redrawn in figures/stage2/.
Commit(s)	feat(ui): give StoryGUI a deep blue-purple palette with layered shades

Original ID	F02, F03, F04, F07, F09: Lore and characters move to a collapsible sidebar; the recap becomes a card
Original (Stage 1)	the report names what the GUI offers (lore and character management, the lore used in a turn, proposals, the recap) but not where it sits. Up to week 3 StoryGUI had three tabs (the story, Lore, Characters), a "Lore used in the last turn" list under the transcript, the proposals at the bottom of the Lore tab and the recap in a titled box with a Hide button at the top of the story tab.

Implemented	the tabs are gone. An open story is one screen (StoryGUI.buildStoryCard): a heading with the story's title (storyHeading; the whole title is also its tooltip), the story column (recap, transcript, Undo / Retry / Edit / End story, input) and, on the right, a sidebar 400 px wide (buildSidebar). The sidebar holds a "Lore used this turn" card (a note when the last turn looked nothing up) above three tabs: Lore, Characters and Proposals (sidebarTabs, LORE_TAB, CHARACTERS_TAB, PROPOSALS_TAB). The button in the heading collapses the sidebar and brings it back (setSidebarOpen, isSidebarOpen); collapsed, it reads "« Lore & characters (2)" with the number of waiting proposals. Double-clicking a used entry opens it in the Lore tab and reopens a collapsed sidebar (openLoreEntry). Because the sidebar is narrow, each tab stacks its list above its form, the buttons are New (on the left, apart) and Save / Delete (on the right), the relationships box is titled "Relationships (one-way)", and the proposals have their own tab (titled "Proposals (N)") with Approve and Reject enabled only while something is waiting. The recap is a StoryTheme.Card above the transcript (buildRecapCard): a raised shade with an accent stripe and a small "Story recap" heading, the text with no box around it, as tall as the text up to RECAP_ROWS (5) lines, and a quiet Dismiss link.
What Changed and Why	the user asked for lore and characters in their own collapsible sidebar on the right, including the lore used this turn, and for the recap to stand out as a layer instead of looking like another menu. With the sidebar the lore and the cast stay visible while the story is written, instead of being a tab away.
How the User Need Is Still Met	every control of F02, F03, F04, F07, F09 and F10 is still in the GUI and calls the same StoryController methods; only the layout changed. The hidden pivotal events are still never shown. The CLI is unchanged.
Affected Use Cases / Classes / Methods / Patterns	UC02, UC03, UC04, UC05 (review a proposal), UC07 (the recap), the display steps only; StoryGUI (buildStoryCard, buildStoryColumn, buildRecapCard, buildSidebar, setSidebarOpen, isSidebarOpen, openLoreEntry, refreshProposals, refreshStory, savedColour; tabs and TAB_TITLE_LENGTH removed), StoryTheme.Card; MVC (view only), Observer (the sidebar is refreshed from onStoryChanged like the old tabs).
Diagrams Updated	figures/class-diagram/02-ui.png (the sidebar methods); redrawn in figures/stage2/.
Commit(s)	feat(ui): move lore, characters and used lore into a collapsible sidebar; feat(ui): set New apart in the sidebar editors and rename the recap heading

5. AI Tools and Collaboration Log

Every AI tool used while building the project, what it was used for, and its token and money cost. Costs are API-equivalent estimates at Anthropic's published per-token rates; sessions run through a subscription may have cost $0 in practice.

5.1 Tools and Models Used

Tool	Model	Used for	Tokens (approx.)	Cost (approx.)
Claude Code	Claude Opus 5.5	Project scaffolding; Stage 2 report scaffold; week 1 (F02, F03, F11, Adapter, controller, both front ends); week 2 (F04 tool-use turn, F05 hidden events, F01 scenario draft, F07 lore proposals, F10 undo/retry/edit, GUI story tab, redrawn UML diagrams); week 3 (F06 plot steering, F08 consistency check, F09 summaries and recap, F12 endings, redrawn class and sequence diagrams); Nov 9 GUI polish (gradual text reveal, start screen); Nov 14 report completion (this report's remaining sections, the redrawn overview diagram, README).	~86M input incl. cache reads, ~1.0M output	~$42.50
Claude Code	Claude Fable 5.2	GUI polish on Nov 9: lore, characters, proposals and the lore used this turn in a collapsible sidebar, the blue-purple palette (StoryTheme), the recap card, the redrawn ui class diagram	~7M input incl. cache reads, ~85K output	~$8.50

Tool	Model	Used for	Tokens (approx.)	Cost (approx.)
The app itself (development and recording)	Claude Sonnet 5.5	Real API calls made while developing and recording the app	*[TO ADD (Shane): input and output tokens from the Anthropic Console usage page]*	*[TO ADD (Shane): cost from the Anthropic Console]*

5.2 Cost Summary

Development with AI tools came to about $51 at API-equivalent rates: about $42.50 for seven Claude Code sessions on Claude Opus 5.5 (about 86M input tokens, almost all of them cache reads of the session context, and about 1.0M output tokens) and about $8.50 for one session on Claude Fable 5.2 (about 7M input and 85K output tokens). The sessions ran through a Claude subscription, so the money actually charged for them was the subscription, not these per-token amounts. The app's own use of Claude Sonnet 5.5 while developing and recording is in the last row of the table in 5.1.

How the estimates were made: each session's input, cache-read and output tokens were read from the session's token counter or, where it was not available, estimated from the number of model calls and the size of the context they re-read; each row of COST-TRACKING.md says which. They were priced at the rates below. The Fable 5.2 rate is taken as twice the Opus 5.5 rate and was not checked against a published price list.

Model	Input $/1M	Output $/1M	Cache read $/1M
Claude Opus 5.5 (development sessions)	4.00	20.00	0.20
Claude Fable 5.2 (one development session; taken as twice the Opus 5.5 rates)	8.00	40.00	0.40
Claude Sonnet 5.5 (the app's LLM)	2.00	10.00	0.20

5.3 Representative Interactions

1. Scaffolding from the design (Oct 20). Asked to set up the repository from the Stage 2 checklist and the Stage 1 report, Claude Code read the Stage 1 class diagrams and created one skeleton per Stage 1 class in its package, the Maven build, .gitignore and .env.example, and the AGENTS.md rules (Conventional Commits, docs updated in the same commit as the code). Accepted as is; the class names in the code still match the diagrams.

2. A defect caught by a generated test (Oct 20). While building F03, a scripted CLI test that Claude wrote failed because a relationship kept the target's name as the user typed it ("guildmaster orrin"). Claude

fixed the model (StorySession stores the character's own spelling) rather than loosening the test; the defect is in Section 6.3.

3. A design change proposed during implementation (Nov 3). While building F09, Claude noticed that summarizing at the end of takeTurn would leave the newest turn out of both the summary and the recent-turn window, because the turn is added to the session only after takeTurn returns. It moved the summary to the start of the turn and logged the change (Section 4, "F09: Older turns are summarized"). Accepted.

4. A request that was not followed literally (Nov 9). Shane asked for the text to be paced because he assumed the API returns the whole response at once. Claude pointed out that the Messages API can stream, but recommended pacing the display instead: a turn is a tool-use loop whose narration is checked and committed before it is shown, so streaming would show text that a failed or retried turn takes back. Shane accepted the display pacing (TranscriptPacer).

5. A correction from the user (Nov 9). Claude had assumed a price for Claude Fable 5.2 in the cost log; Shane corrected the rate, and the log and Section 5.2 now say how that rate was chosen.

5.4 How AI Output Was Checked

Every change was made on a feature branch and reached main only through a pull request that Shane reviewed and merged with a merge commit (PRs #1 to #7). Before each commit, mvn test was run; a commit that did not build or whose tests failed was not pushed. Agent and tool code was tested with FakeLLMClient, which scripts the model's replies, including refusals, failed calls and invalid tool requests, so each error path in Section 2 has a test. GUI changes were rendered under Xvfb and checked by screenshot, and the CLI was run with scripted input.

Generated code was also checked against the design: class and method names against the Stage 1 diagrams, and every difference recorded in docs/changes-from-stage1.md when it was made. Examples of corrections: the relationship-name defect above, found by a test; the summary timing in F09, found by reading the turn sequence against SD05 and SD06; existing turn tests that broke when the consistency check added a model call, which were fixed by scripting the extra reply rather than weakening the check.

6. Development and Verification

6.1 Weekly Commit Summary

At least five meaningful code commits were required in each period (Toronto time). Report-only, formatting-only, empty or artificially split commits are not counted.

Period 1: Oct 20 to Oct 26, 2026

#	Commit	Summary	Features
1	64aa840 (https://github.com/Polarralop/EECS3311Project/commit/64aa840)	feat(model): add lore entries with tag, search and pin in LoreBook	F02

#	Commit	Summary	Features
2	a80fa3a (https://github.com/Polarralop/EECS3311Project/commit/a80fa3a)	feat(model): manage characters and relationships in StorySession	F03
3	957b423 (https://github.com/Polarralop/EECS3311Project/commit/957b423)	feat(model): notify StoryObserver on session changes	F04, F05, F07, F09, F10 (model, Observer)
4	0913872 (https://github.com/Polarralop/EECS3311Project/commit/0913872)	feat(persistence): save and load sessions as JSON	F11
5	cee9994 (https://github.com/Polarralop/EECS3311Project/commit/cee9994)	feat(llm): send requests through ClaudeAdapter with tool-use support	F04 (Adapter)
6	c749429 (https://github.com/Polarralop/EECS3311Project/commit/c749429)	feat(controller): expose lore, characters and save/load in StoryCLI	F02, F03, F11 (MVC)
7	2990b8f (https://github.com/Polarralop/EECS3311Project/commit/2990b8f)	feat(ui): manage lore, characters and saves in StoryGUI	F02, F03, F11 (MVC, Observer)
9	63a3cc5 (https://github.com/Polarralop/EECS3311Project/commit/63a3cc5)	feat(ui): open StoryGUI to a story picker and name the tab after the story	F01, F11 (GUI)
10	57725a1 (https://github.com/Polarralop/EECS3311Project/commit/57725a1)	feat(ui): show when each saved story was saved instead of its id	F11 (GUI, CLI)

#	Commit	Summary	Features
11	29010dd (https://github.com/Polarralop/EECS3311Project/commit/29010dd)	feat(ui): give StoryGUI a deep blue-purple palette with layered shades	GUI look (StoryTheme)
12	6af4015 (https://github.com/Polarralop/EECS3311Project/commit/6af4015)	feat(ui): move lore, characters and used lore into a collapsible sidebar	F02, F03, F04, F07, F09 (GUI)
13	31d9c33 (https://github.com/Polarralop/EECS3311Project/commit/31d9c33)	feat(ui): set New apart in the sidebar editors and rename the recap heading	F02, F03, F09 (GUI)

Count: 7 meaningful code commits, all on Oct 20.

Period 2: Oct 27 to Nov 2, 2026

#	Commit	Summary	Features
1	2f31992 (https://github.com/Polarralop/EECS3311Project/commit/2f31992)	feat(tools): register SearchLoreTool and GetCharacterTool in ToolRegistry	F04 (tools)
2	139c4c0 (https://github.com/Polarralop/EECS3311Project/commit/139c4c0)	feat(agent): run the tool-use loop in StoryAgent.takeTurn	F04, F06 (Facade, Strategy)
3	569e476 (https://github.com/Polarralop/EECS3311Project/commit/569e476)	feat(agent): plan hidden pivotal events before the story starts	F01, F05
4	c5a57de (https://github.com/Polarralop/EECS3311Project/commit/c5a57de)	feat(agent): draft a scenario from a short prompt	F01

#	Commit	Summary	Features
5	912b3d0 (https://github.com/Polarralop/EECS3311Project/commit/912b3d0)	feat(controller): submit turns through SubmitTurnCommand and CommandHistory	F04, F10 (Command)
6	f260c7f (https://github.com/Polarralop/EECS3311Project/commit/f260c7f)	feat(controller): edit a past turn with EditTurnCommand	F10 (Command)
7	d86552d (https://github.com/Polarralop/EECS3311Project/commit/d86552d)	feat(tools): raise lore proposals and resolve them in StoryController	F07
8	70e760d (https://github.com/Polarralop/EECS3311Project/commit/70e760d)	feat(ui): play turns and review proposals in StoryGUI	F04, F07, F10, F01 (GUI)

Count: 8 meaningful code commits in period 2, all on Oct 27.

Period 3: Nov 3 to Nov 9, 2026

#	Commit	Summary	Features
1	53dcf44 (https://github.com/Polarralop/EECS3311Project/commit/53dcf44)	feat(tools): steer the hidden plot with UpdatePlotTool	F06
2	bf3e57d (https://github.com/Polarralop/EECS3311Project/commit/bf3e57d)	feat(agent): flag contradictions with pinned lore in ConsistencyChecker	F08
3	d80ffd1 (https://github.com/Polarralop/EECS3311Project/commit/d80ffd1)	feat(agent): summarize older turns in MemoryManager	F09

#	Commit	Summary	Features
4	62b5881 (https://github.com/Polarralop/EECS3311Project/commit/62b5881)	feat(controller): show a model-written recap when a story is loaded	F09, F11
5	0de62b7 (https://github.com/Polarralop/EECS3311Project/commit/0de62b7)	feat(agent): write the story's ending in StoryAgent.generateEnding	F12
6	c242f87 (https://github.com/Polarralop/EECS3311Project/commit/c242f87)	feat(ui): end the story from StoryGUI and StoryCLI	F12 (GUI, CLI)
7	04dae82 (https://github.com/Polarralop/EECS3311Project/commit/04dae82)	feat(ui): reveal new story text gradually in both front ends	F04, F10, F12 (GUI, CLI)
8	ff829a3 (https://github.com/Polarralop/EECS3311Project/commit/ff829a3)	feat(controller): list saved stories by title and open the CLI to them	F11 (CLI)
9	63a3cc5 (https://github.com/Polarralop/EECS3311Project/commit/63a3cc5)	feat(ui): open StoryGUI to a story picker and name the tab after the story	F01, F11 (GUI)
10	57725a1 (https://github.com/Polarralop/EECS3311Project/commit/57725a1)	feat(ui): show when each saved story was saved instead of its id	F11 (GUI, CLI)
11	29010dd (https://github.com/Polarralop/EECS3311Project/commit/29010dd)	feat(ui): give StoryGUI a deep blue-purple palette with layered shades	GUI look (StoryTheme)

#	Commit	Summary	Features
12	6af4015 (https://github.com/Polarralop/EECS3311Project/commit/6af4015)	feat(ui): move lore, characters and used lore into a collapsible sidebar	F02, F03, F04, F07, F09 (GUI)
13	31d9c33 (https://github.com/Polarralop/EECS3311Project/commit/31d9c33)	feat(ui): set New apart in the sidebar editors and rename the recap heading	F02, F03, F09 (GUI)

Count: 13 meaningful code commits in period 3 (6 on Nov 3, 3 on Nov 8 and 4 on Nov 9, Toronto time). Every period has at least five; report-only commits are not counted in any period.

6.2 Checks Performed

Check	How	Result
Unit tests	mvn test (JUnit, FakeLLMClient; no paid calls)	260 tests, all passing (Nov 9, 2026; re-run on Nov 14 with the same result). Agent, tool, controller and front-end tests script the model with FakeLLMClient; ClaudeAdapterTest runs the adapter against a local HTTP server that plays the Messages API; no test makes a paid call.

Check	How	Result
Build from a fresh clone	Follow README.md on a clean checkout	Nov 14: a fresh clone of main (c4b0413) was built by following README.md (cp .env.example .env, mvn compile). mvn exec:java@cli started on "No stories detected", and help, sessions and the README example (copy samples/sample-ashfall.json to saves/, load sample-ashfall, lore list, char show Mira) gave the expected output. mvn exec:java@gui opened without errors under Xvfb. No key was needed for these steps; story turns need one.
GUI walk-through	mvn compile exec:java@gui, every feature	Oct 20: Lore and Characters tabs checked on screen with the sample story. Oct 27: Story tab, Start story form and proposals list rendered under Xvfb with a scripted model and checked by screenshot; StoryGUITest drives every button headless (start, send, undo, retry, edit, approve, reject). Nov 9: start screen (no stories, then Select story), the story-titled tab and the gradual reveal of a turn checked on screen under Xvfb with scripted model replies. Nov 9 (sidebar and palette): the start screen, the story view with the recap card, each sidebar tab, the collapsed sidebar and the Start story form rendered under Xvfb with a scripted model and checked by screenshot.

Check	How	Result
CLI walk-through	mvn compile exec:java@cli, every feature	Oct 20: new, lore add/list, save and sessions run through mvn compile exec:java@cli. Oct 27: new, lore add, start and do run through mvn compile exec:java@cli without a key (the start reports the missing key; the app keeps working); StoryCLITest scripts every F01, F04, F05, F07 and F10 command with FakeLLMClient.
Live model run	A real story with Claude Sonnet 5.5	*[TO ADD (Shane): date, and that a story was started, played, saved and loaded with the live model, as recorded in the videos]*
Secrets check	No API key in source, history, logs, screenshots or videos	Nov 14: .gitignore lists .env, .env.* (except .env.example) and config.local.json, and was committed before any code (5cbd9f4). git ls-files shows no .env or config.local.json; .env.example holds only placeholders. A search of every commit in the history for Anthropic key patterns (sk-ant-...) and for an assigned ANTHROPIC_API_KEY value found none. The screenshots use scripted replies and show no key.

6.3 Defects Found and Fixed

Defect	How found	Fix	Commit
A relationship stored the target's name as typed (for example "guildmaster orrin"), so it did not match the character's real spelling.	StoryCLITest.characterAndRelationshipCommands	StorySession now stores the target character's own spelling when a relationship is checked.	c749429 (https://github.com/Polarralop/EECS3311Project/commit/c749429)
A summary written at the end of a turn would have left the newest turn out of both the running summary and the recent-turn window, because the turn is added to the session only after StoryAgent.takeTurn returns.	Reading the turn sequence (SD05, SD06) against the code while building F09	MemoryManager.summarizeIfNeeded runs at the start of the next turn instead; MemoryManagerTest.theAgentSummarizesBeforeBuildingTheTurnsContext covers it.	d80ffd1 (https://github.com/Polarralop/EECS3311Project/commit/d80ffd1)

6.4 Limitations and Stage 3 Work

The limitations in Section 1.5 remain: the storytelling quality is only checked by hand with the live model; turns are displayed after they finish rather than streamed; the consistency check and plot steering depend on the model's judgement; early turns survive only as a summary; undo history is not saved; saves are local and single-user.

Stage 3 work, as the project instructions set out: full deterministic testing of the deterministic features (F02, F03, F11 and the deterministic parts of F01, F04, F07 and F10), and KUMA validation of the agent's behaviour (tool use, planning, memory and consistency checking) against the live model.

7. Demonstration Index

Every implemented feature has a video or a labelled segment. Each shows the feature ID and name, how it is invoked and its input, the running app and its output, where AI is involved, and error or alternative behaviour where relevant. Links open without an access request, and no API key is visible.

Feature	Front end	Video link	Timestamp	What is shown
F01: Scenario Setup	*[GUI / CLI]*	*[TO ADD: link]*	*[mm:ss]*	*[TO ADD: what the segment shows]*
F02: Lore Entry Management	*[GUI / CLI]*	*[TO ADD: link]*	*[mm:ss]*	*[TO ADD: what the segment shows]*
F03: Character Management	*[GUI / CLI]*	*[TO ADD: link]*	*[mm:ss]*	*[TO ADD: what the segment shows]*

Feature	Front end	Video link	Timestamp	What is shown
F04: Story Turn	*[GUI / CLI]*	*[TO ADD: link]*	*[mm:ss]*	*[TO ADD: what the segment shows]*
F05: Pivotal Events	*[GUI / CLI]*	*[TO ADD: link]*	*[mm:ss]*	*[TO ADD: what the segment shows]*
F06: Plot Steering	*[GUI / CLI]*	*[TO ADD: link]*	*[mm:ss]*	*[TO ADD: what the segment shows]*
F07: Lore Proposals	*[GUI / CLI]*	*[TO ADD: link]*	*[mm:ss]*	*[TO ADD: what the segment shows]*
F08: Consistency Check	*[GUI / CLI]*	*[TO ADD: link]*	*[mm:ss]*	*[TO ADD: what the segment shows]*
F09: Story Memory Summarization	*[GUI / CLI]*	*[TO ADD: link]*	*[mm:ss]*	*[TO ADD: what the segment shows]*
F10: Undo / Retry / Edit Turn	*[GUI / CLI]*	*[TO ADD: link]*	*[mm:ss]*	*[TO ADD: what the segment shows]*
F11: Save and Load Session	*[GUI / CLI]*	*[TO ADD: link]*	*[mm:ss]*	*[TO ADD: what the segment shows]*
F12: Ending Generation	*[GUI / CLI]*	*[TO ADD: link]*	*[mm:ss]*	*[TO ADD: what the segment shows]*